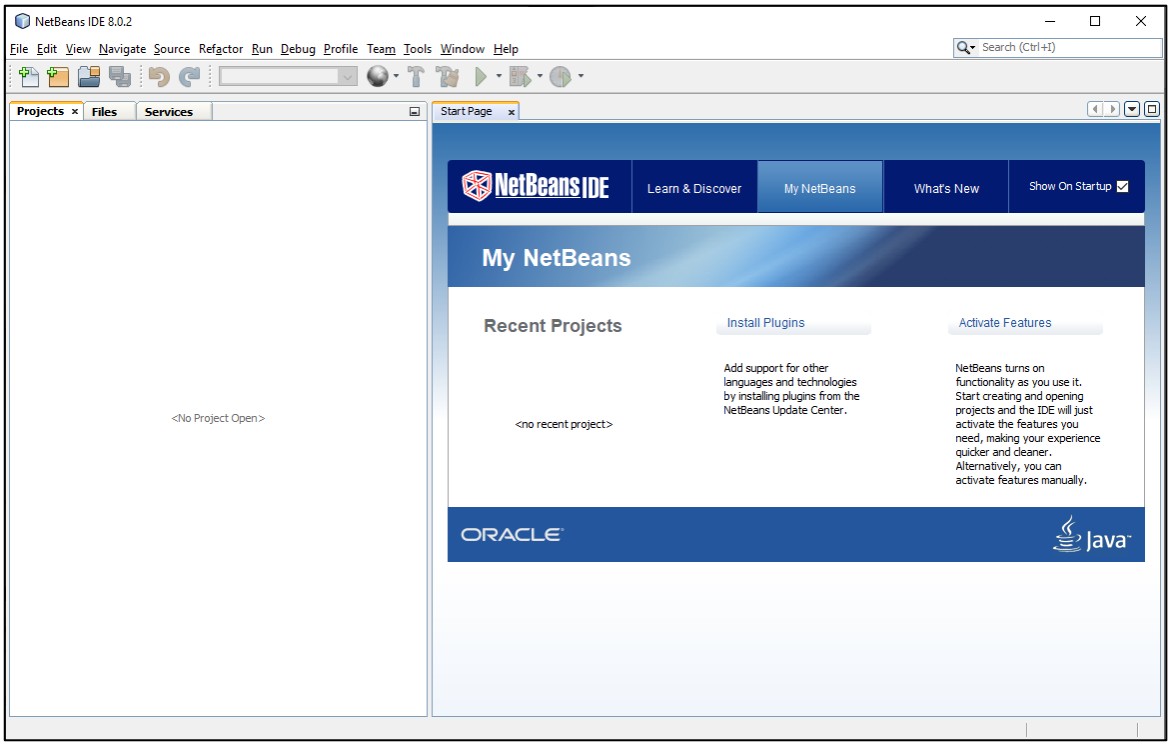
- CC Practical 1A –

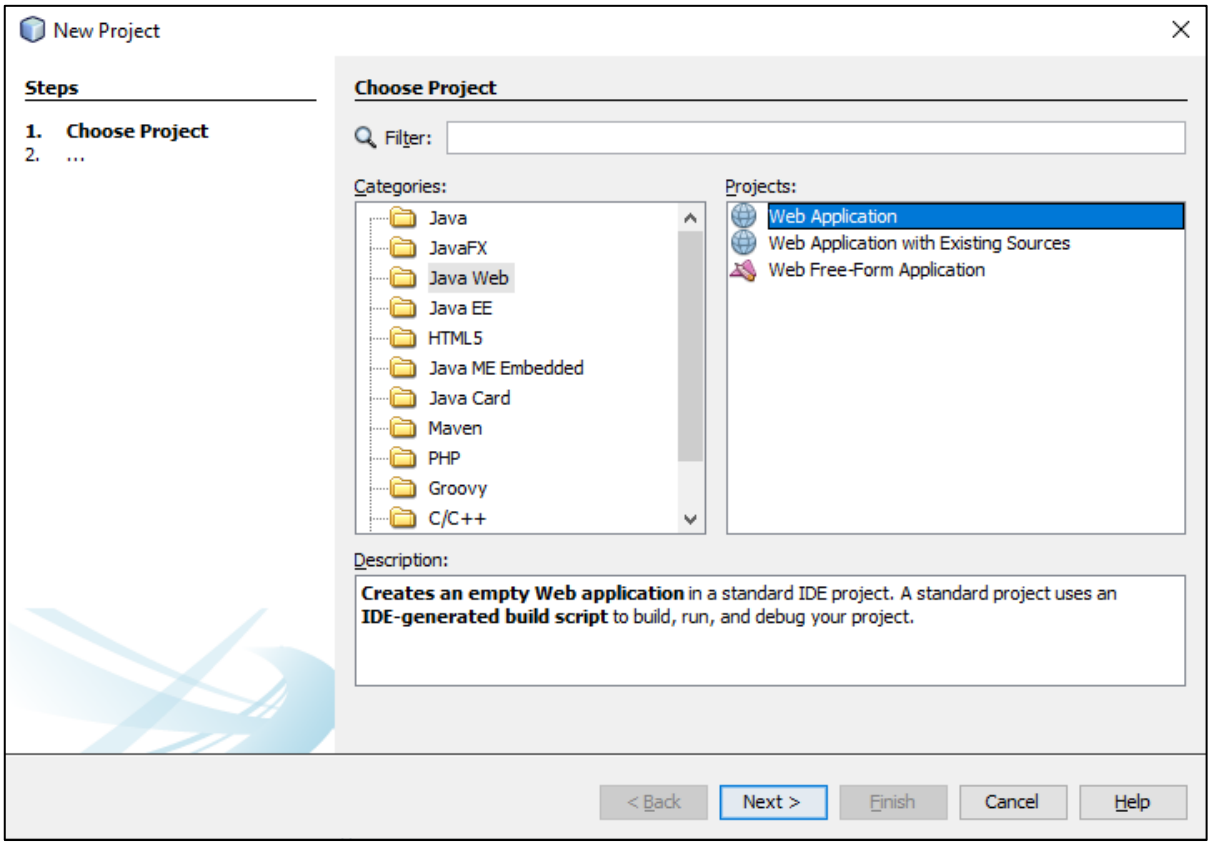
Aim: *Define a Simple SOAP Service like Converting Rs into Dollars and Call it from different Platforms.*

STEPS:

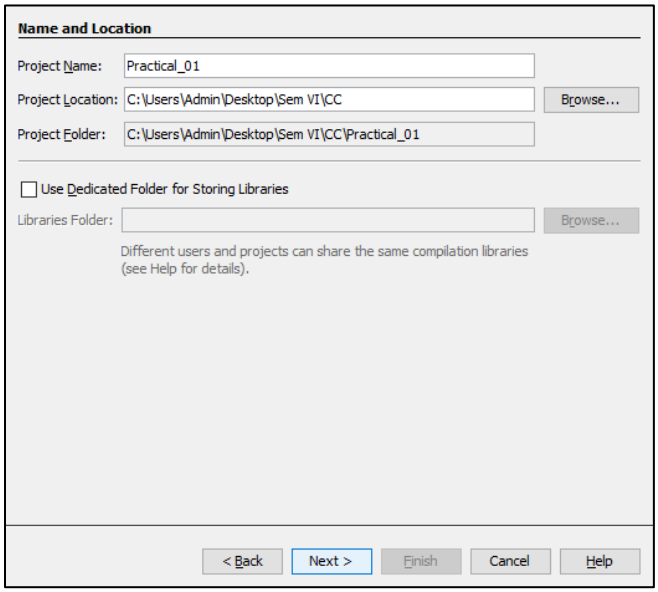
1) Open Netbeans.

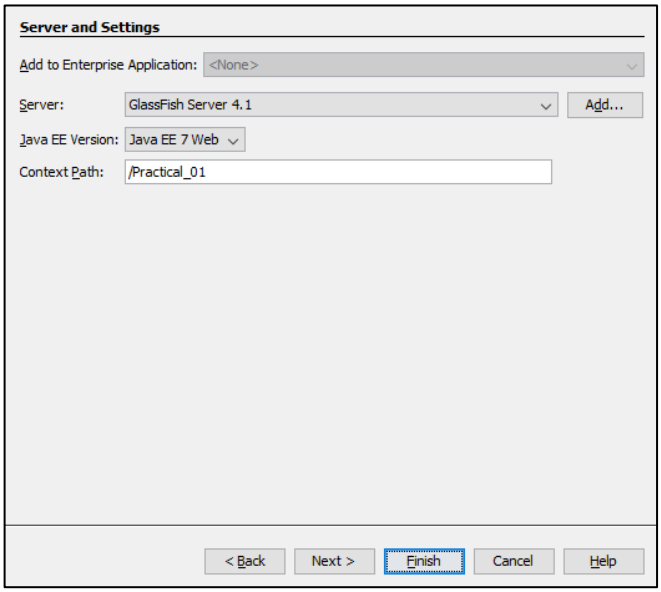


2) Create New Project. Select 'Java Web' from Categories & 'Web Application' from Projects. Click Next.

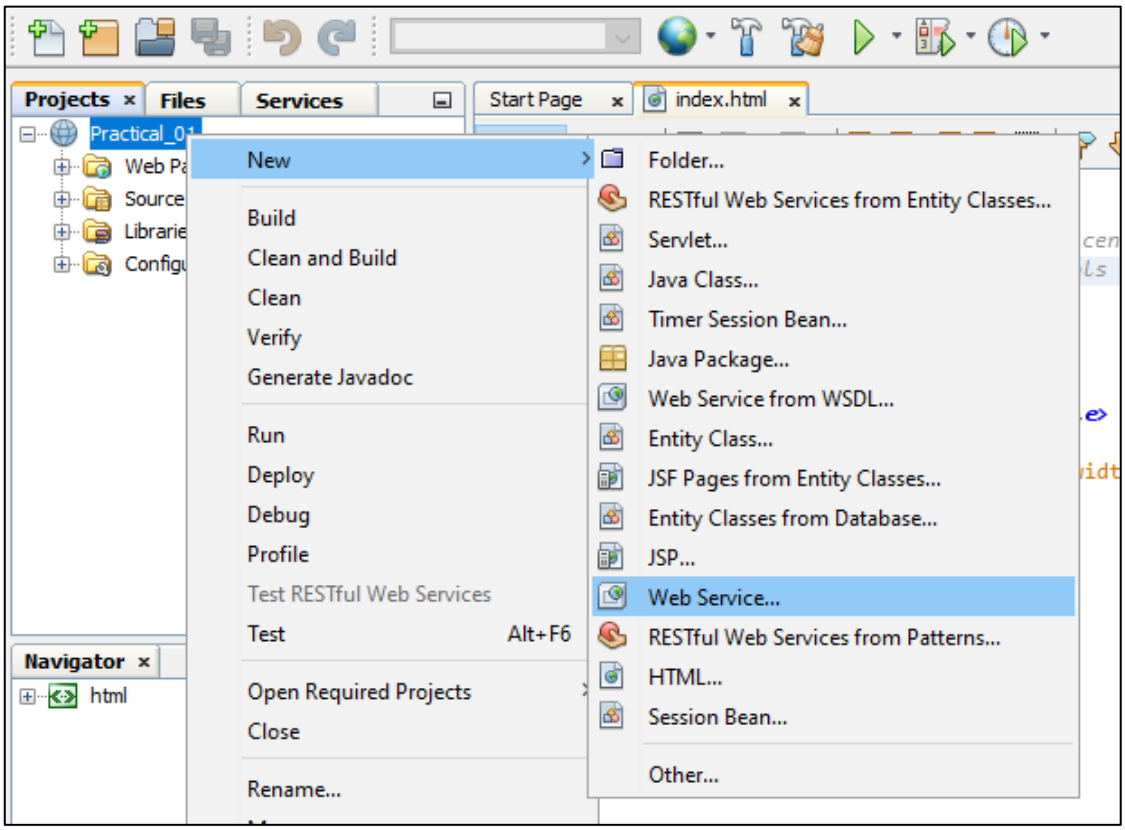


3) Name your Project and choose Location then Click Next, Ensure Glassfish Server is Selected then Click Finish.

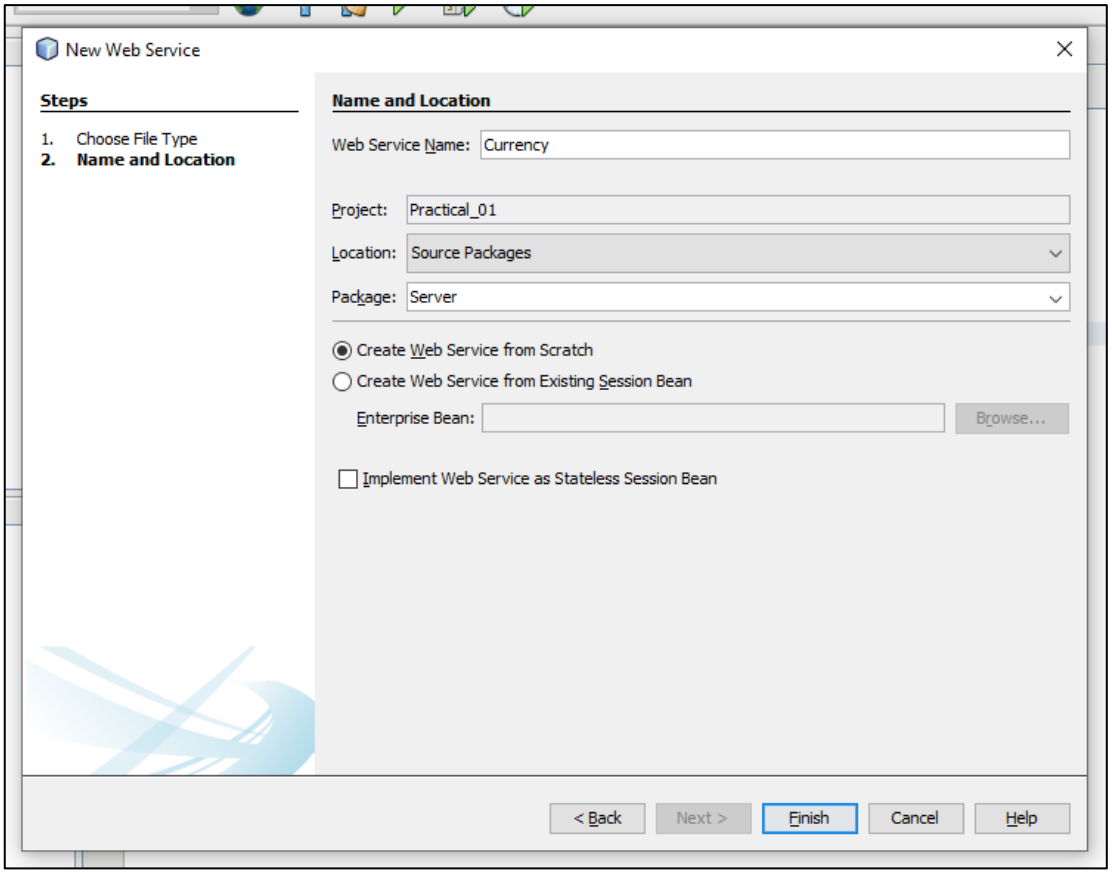




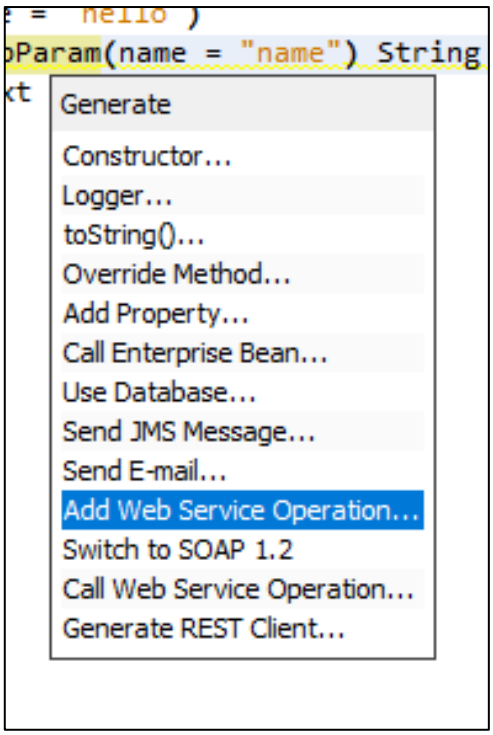
4) Expand your project by clicking on '+' symbol, Select: **New → Web Service.**



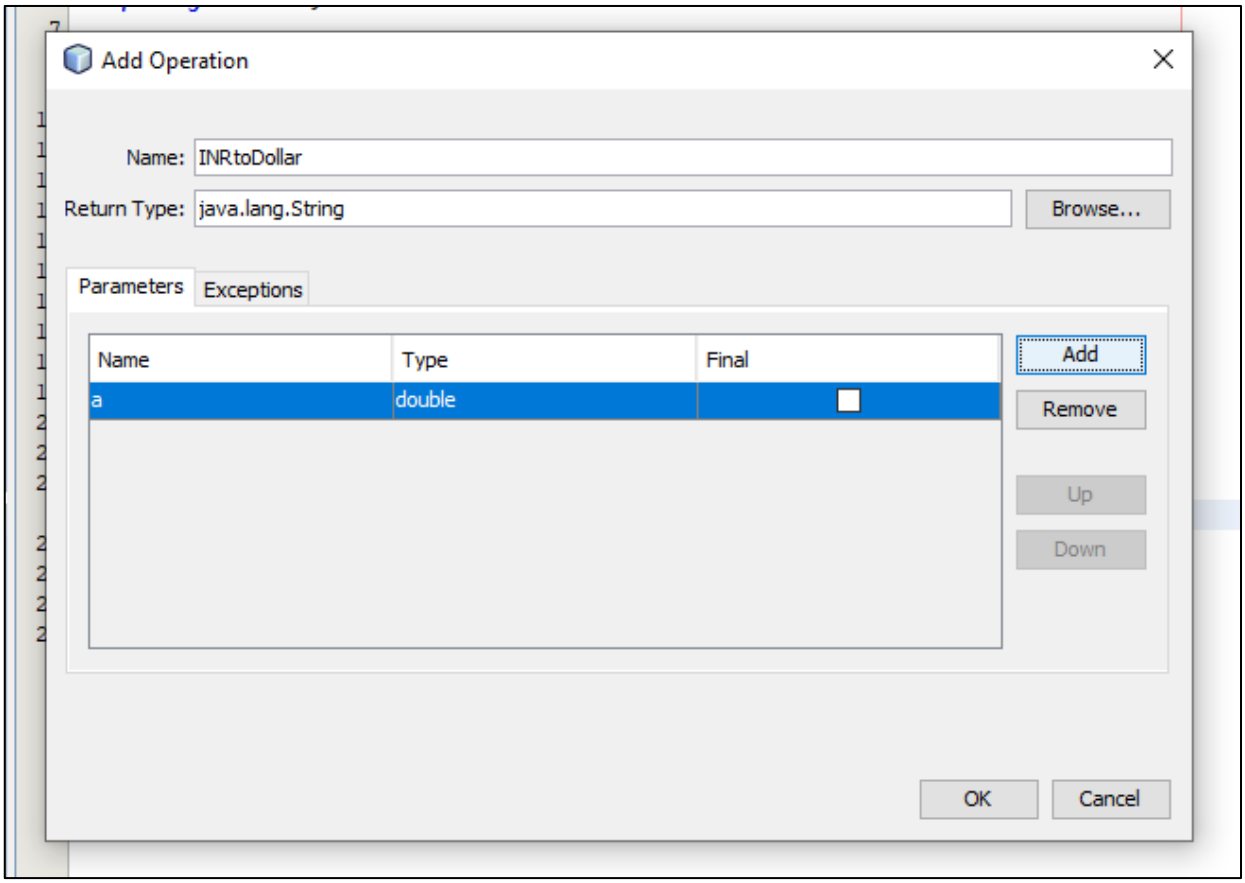
5) Name your Service and type ‘server’



6) Erase Sample Code and Right-Click any where inside the code. Insert Code and select Add Web Service Operation within.



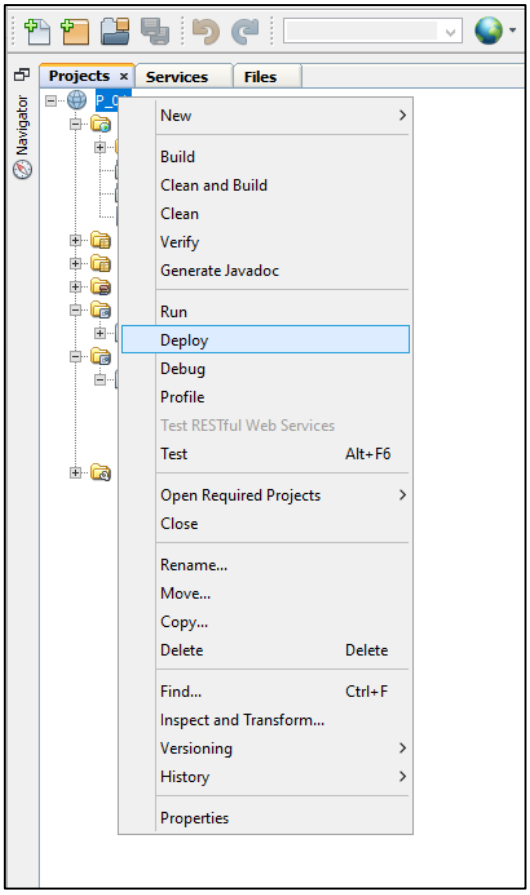
7) In the dialog Box, Name your Method/Operation & Add a paramter. Name it and select it’s type.



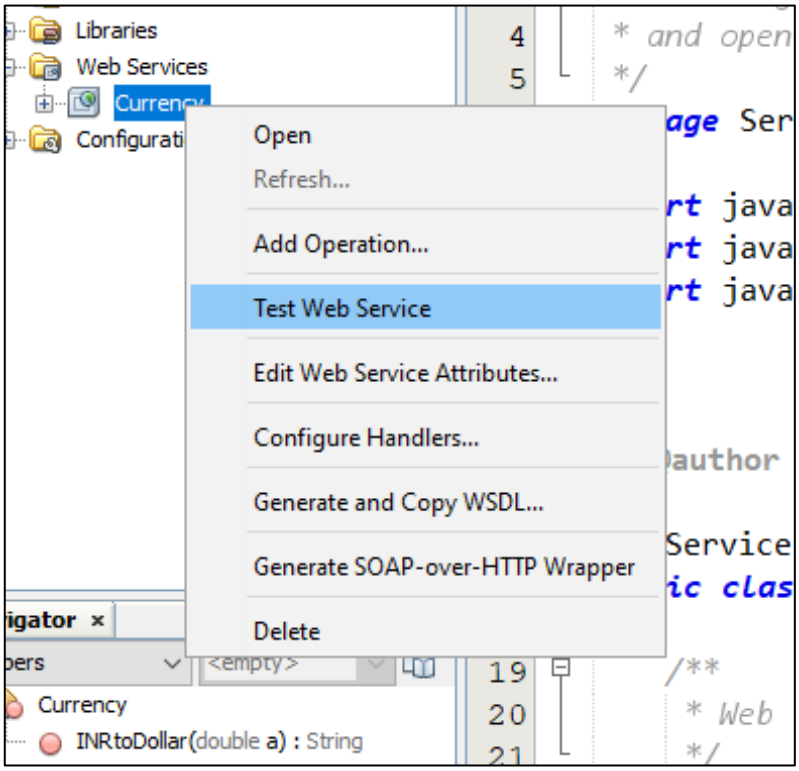
8) In the auto-generated Code, do the following Changes.

```
package Server;

import javax.jws.WebService;
import javax.jws.WebMethod;
import javax.jws.WebParam;

/**
 *
 * @author Admin
 */
@WebService(serviceName = "Currency")
public class Currency {

    /**
     * Web service operation
     */
    @WebMethod(operationName = "INRtoDollar")
    public String INRtoDollar(@WebParam(name = "a") double a) {
        //TODO write your implementation code here:
        return "The Indian Rupees " + a + " in Dollars is " + (a/83.17);
    }
}
```

9) Deploy Project.



10) Click on your Web Service and Test it.



Output prompting Input to Convert Currency (INR to Dollar).

Currency Web Service Tester

This form will allow you to test your web service implementation (WSDL File)

To invoke an operation, fill the method parameter(s) input boxes and click on the button labeled with the method name.

Methods :

public abstract java.lang.String server.Currency.inRtoDollar(double)

inRtoDollar (6520)

Final Output

inRtoDollar Method invocation

Method parameter(s)

Type	Value
double	6520

Method returned

java.lang.String : "**The Indian Rupees 6520.0 in Dollars is 78.39365155705183**"

SOAP Request


```
<?xml version="1.0" encoding="UTF-8"?><S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/" xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
    <SOAP-ENV:Header/>
    <S:Body>
        <ns2:INRtoDollar xmlns:ns2="http://Server/">
            <a>6520.0</a>
        </ns2:INRtoDollar>
    </S:Body>
</S:Envelope>
```


SOAP Response

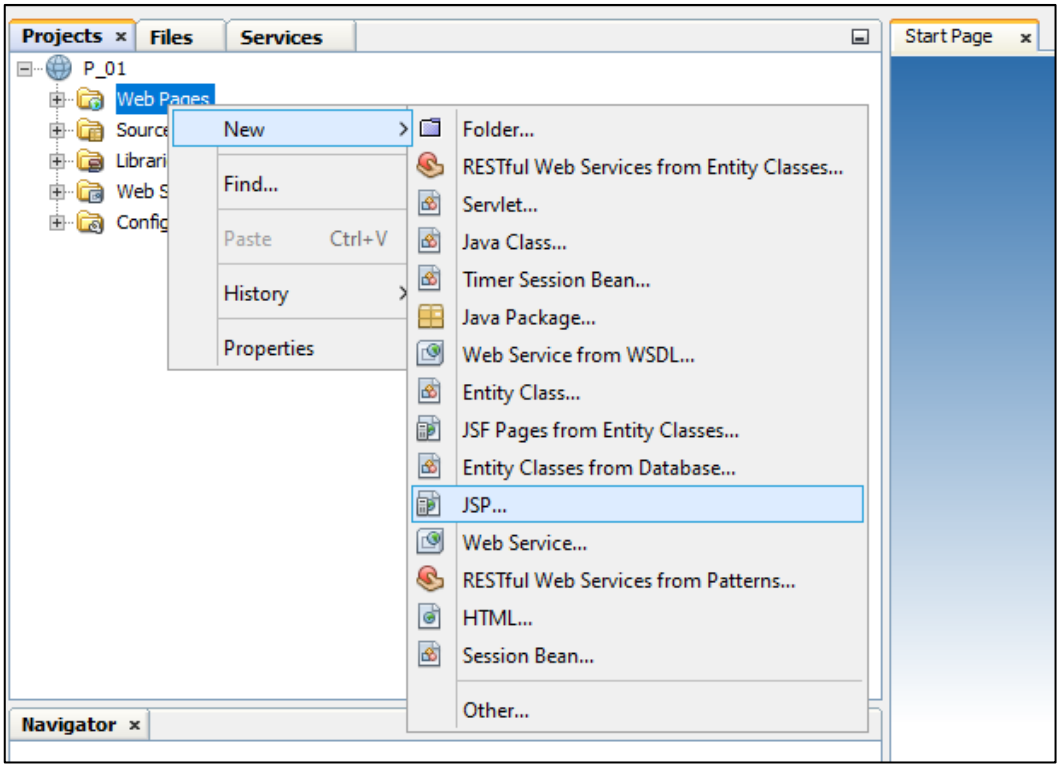

```
<?xml version="1.0" encoding="UTF-8"?><S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/" xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
    <SOAP-ENV:Header/>
    <S:Body>
        <ns2:INRtoDollarResponse xmlns:ns2="http://Server/">
            <return>The Indian Rupees 6520.0 in Dollars is 78.39365155705183</return>
        </ns2:INRtoDollarResponse>
    </S:Body>
</S:Envelope>
```

- CC Practical 1B –

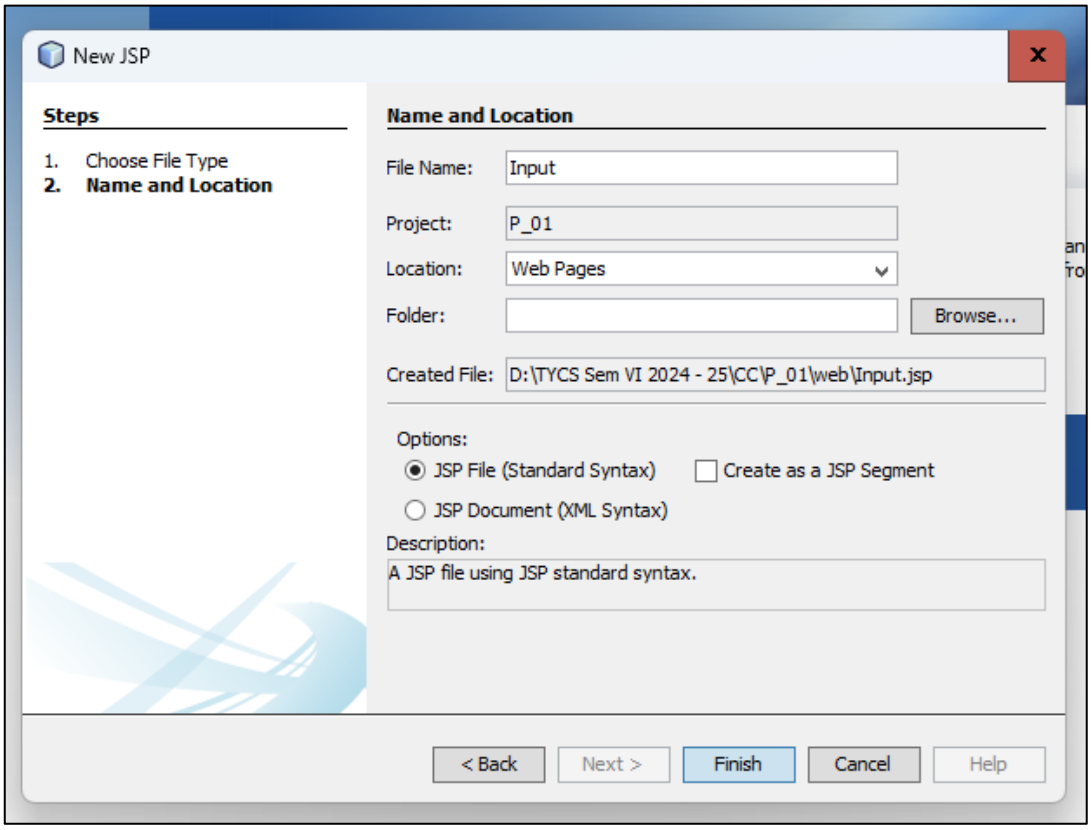
Aim: *Creating JSP Client*

STEPS:

Step 1: Create JSP (Java Server Page).



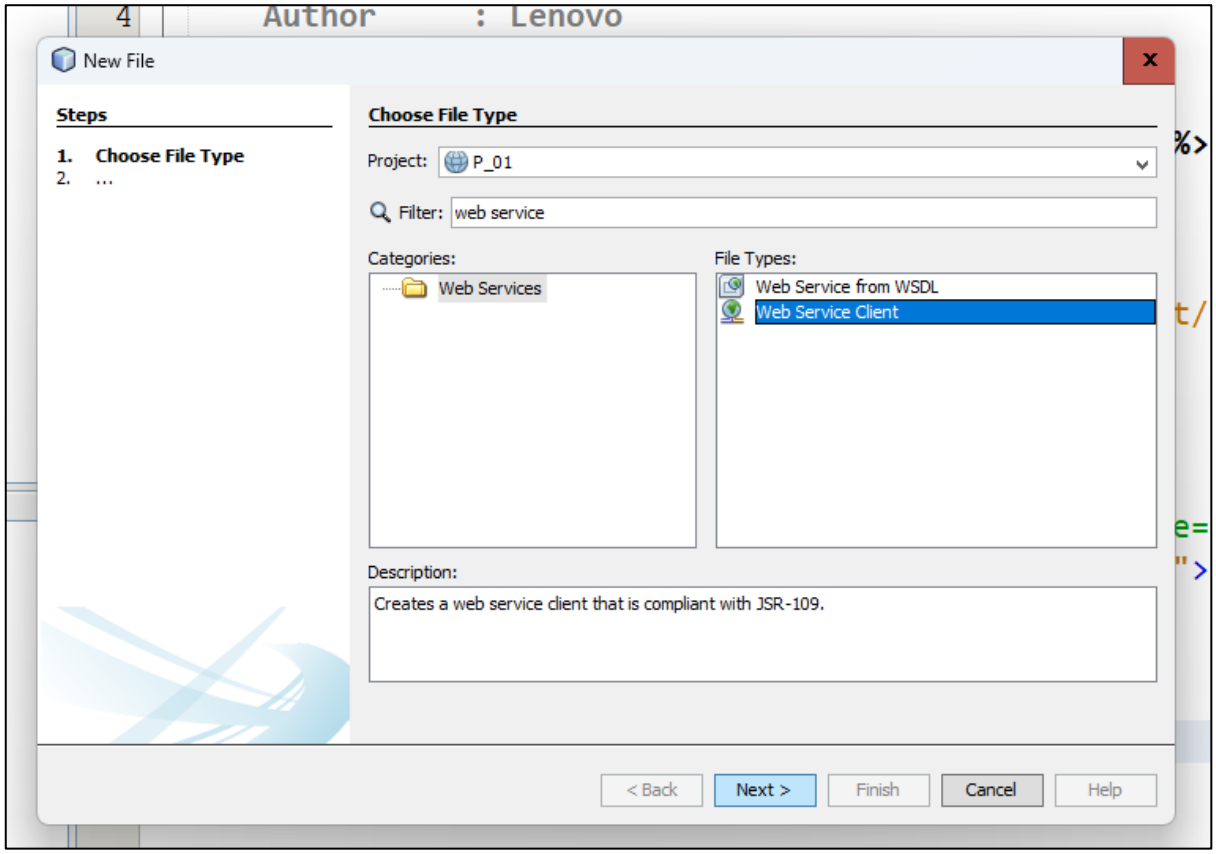
Step 2: Enter File Name and Click on Finish.



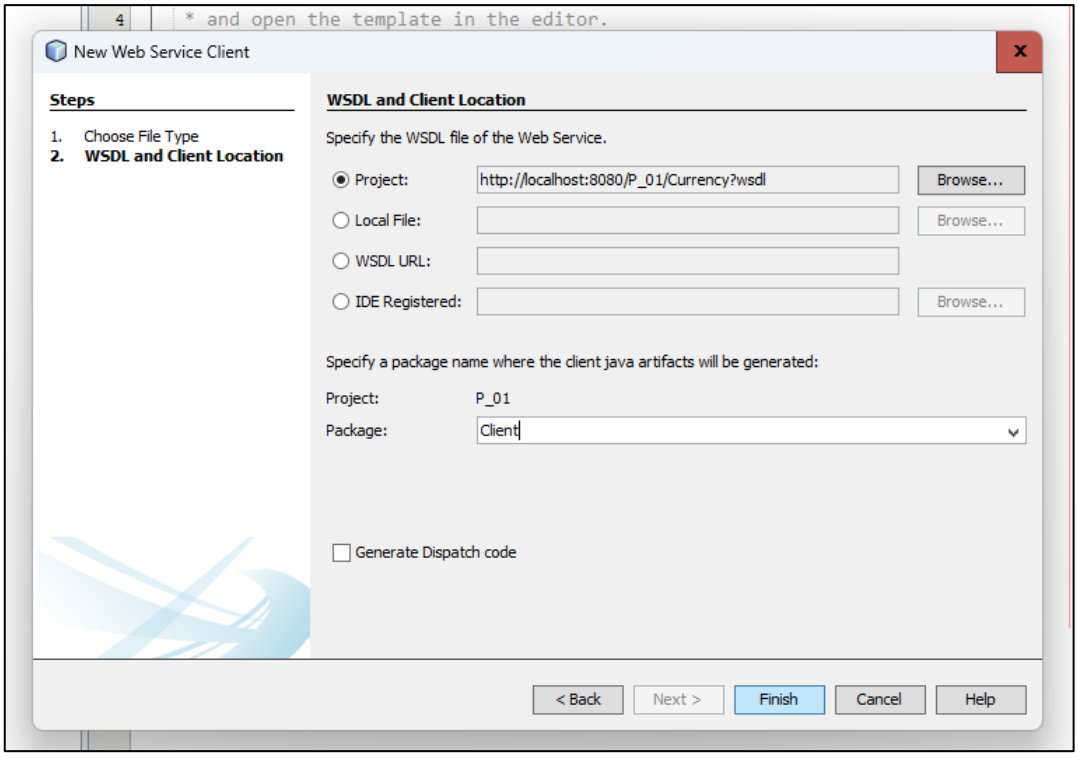
Step 3: Write following Code in “Input” File.

```jsp
<%--
    Document   : Input
    Created on : 13 Dec, 2024, 7:37:39 AM
    Author     : Lenovo
--%>

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>JSP Page</title>
    </head>
    <body>
        <form action="Output.jsp">
            Enter the Currency in Rupees: <input type="text" name="t1">
            <input type="submit"> <input type="reset">
        </form>
    </body>
</html>
```

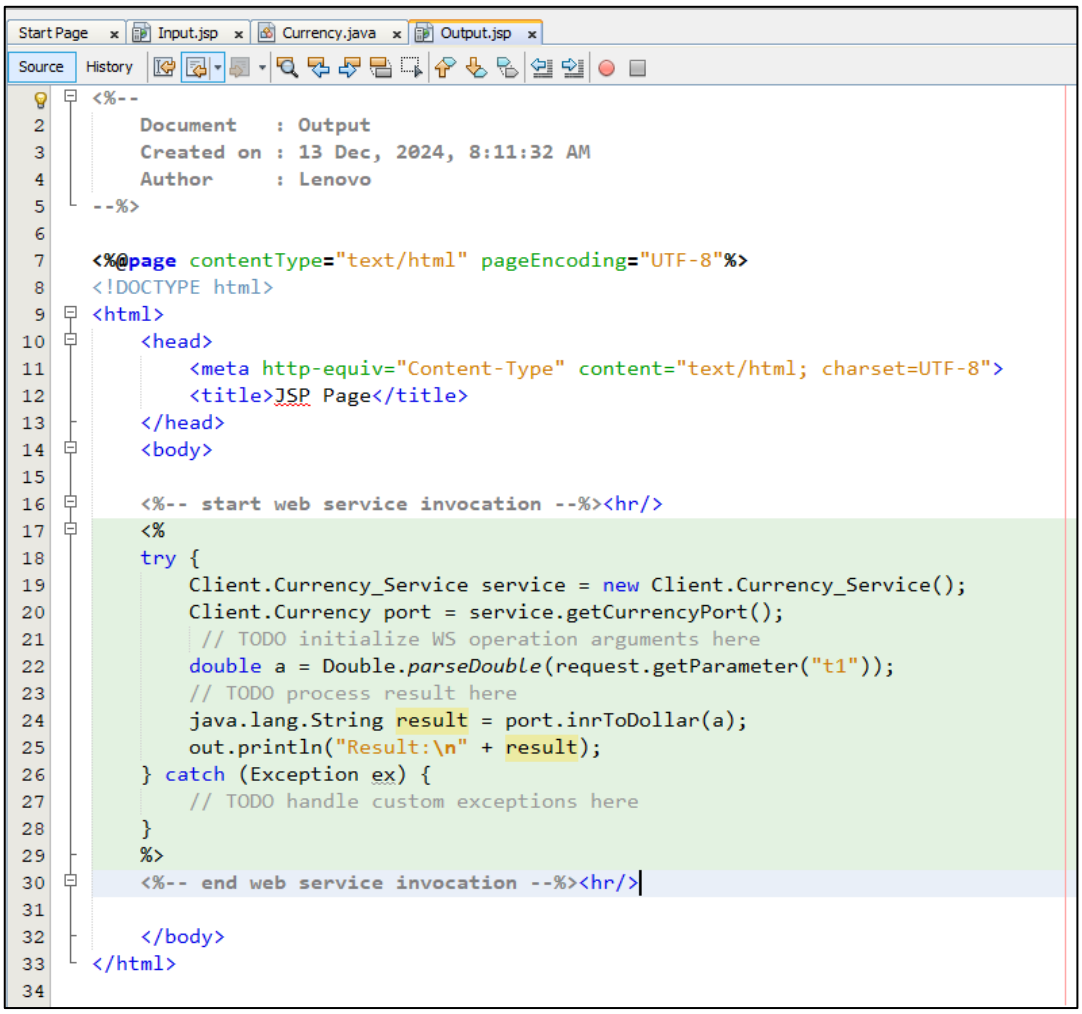
Step 4: Create Web Service Client for the Project.



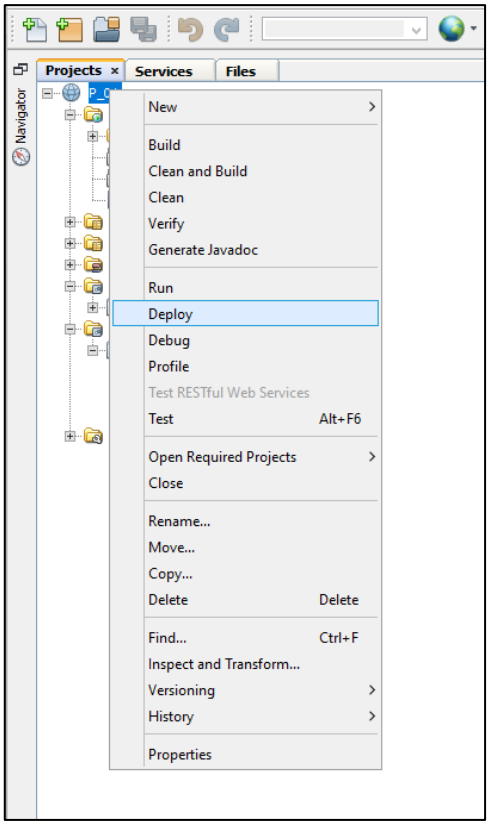
Step 5: Specify Project URL and Type Package as “Client”.



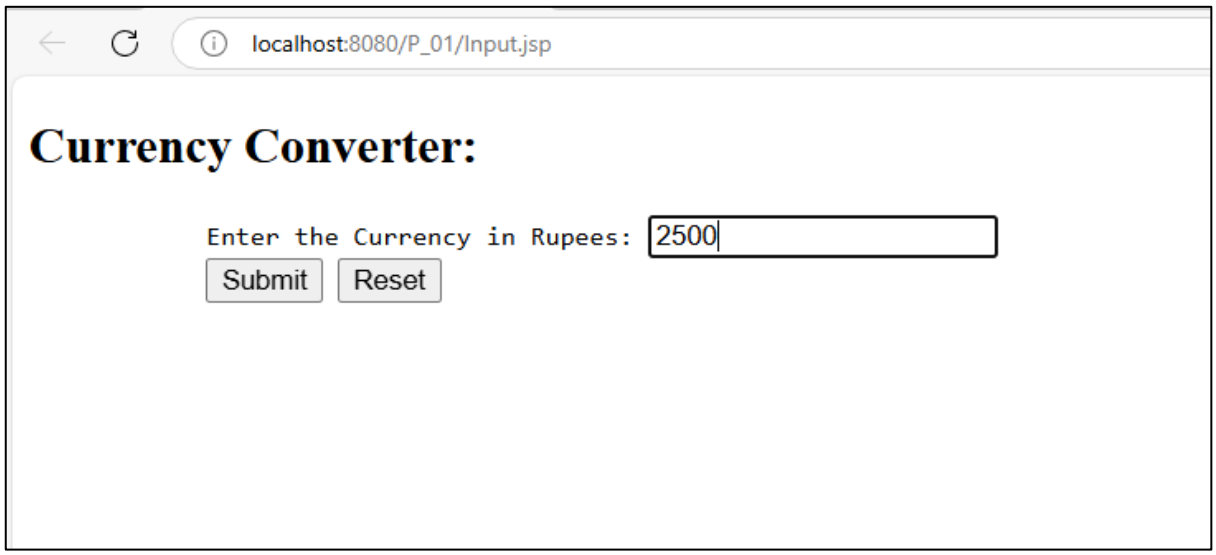
Step 6: Create second JSP, name it as Output and Write the following Code.



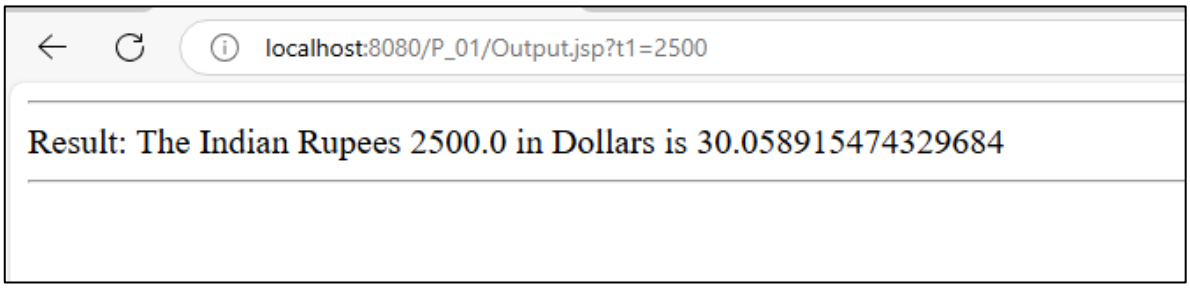
Step 7: Deploy Project and Run the “Input” file.



Output prompting Input to Convert Currency (INR to Dollar).



Final Output



- CC Practical 1C –

Aim: *Creating Python Client*

STEPS:

Step 1: Install Zeep Module and Implement the Code.

P_03.py - D:/TYCS Sem VI 2024 - 25/CC/P_01/P_03.py (3.12.6)

File Edit Format Run Options Window Help

```python
from zeep import Client

wsdl_url = 'http://localhost:8080/P_01/Currency?wsdl'
client = Client(wsdl_url)

# Call Method on Web-Service
print("Currency Converter:\n")
d = float(input("Enter Currency in Indian Rupees: "))
result = client.service.INR_to_Dollar(d)
print(result)
```

Final Output

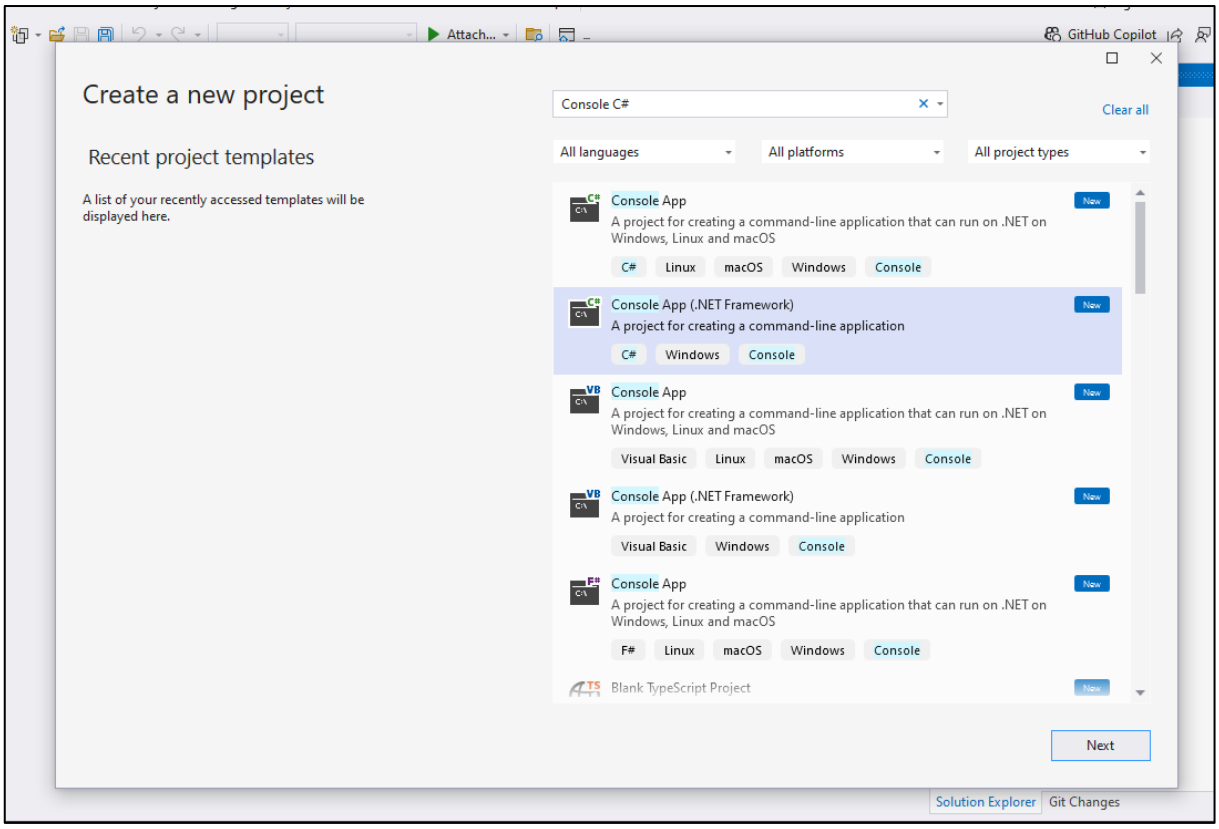
```
>>>
    =============== RESTART: D:/TYCS Sem VI 2024 - 25/CC/P_01/P_03.py ==============
    Currency Converter:

    Enter Currency in Indian Rupees: 2500
    The Indian Rupees 2500.0 in Dollars is 30.058915474329684
>>>
```

- CC Practical 1D –

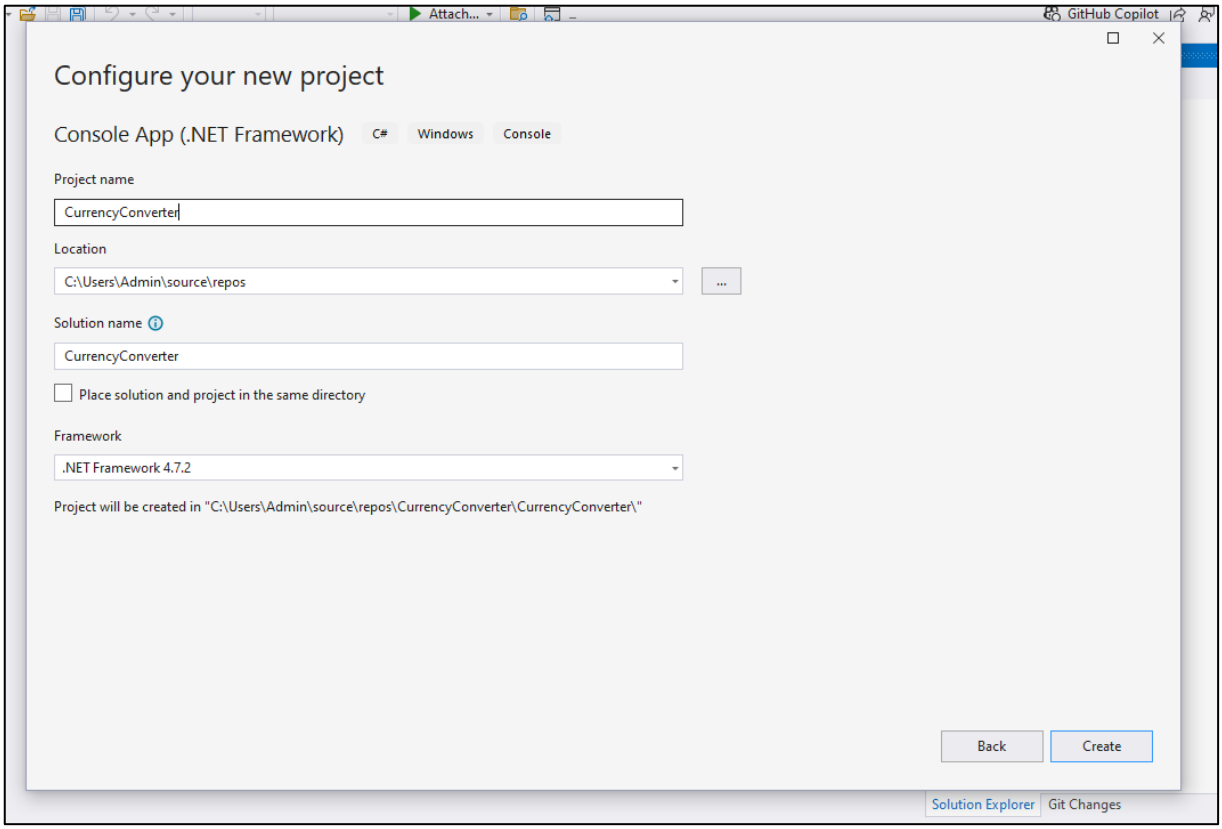
Aim: *Creating .NET Client*

STEPS:

Step 1: Open Visual Studio and Create a New Project. Search for “Console C#” then select “Console App” for .NET freamwork.



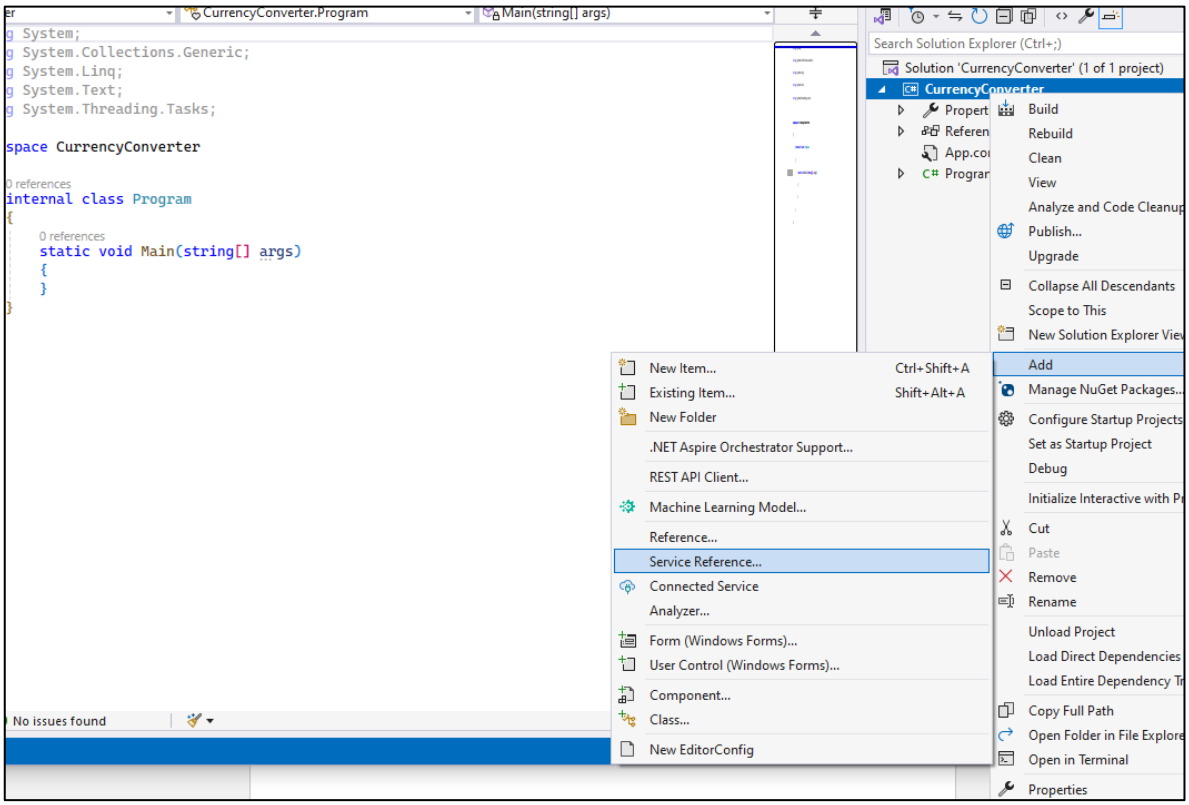
Step 2: Name the Project.



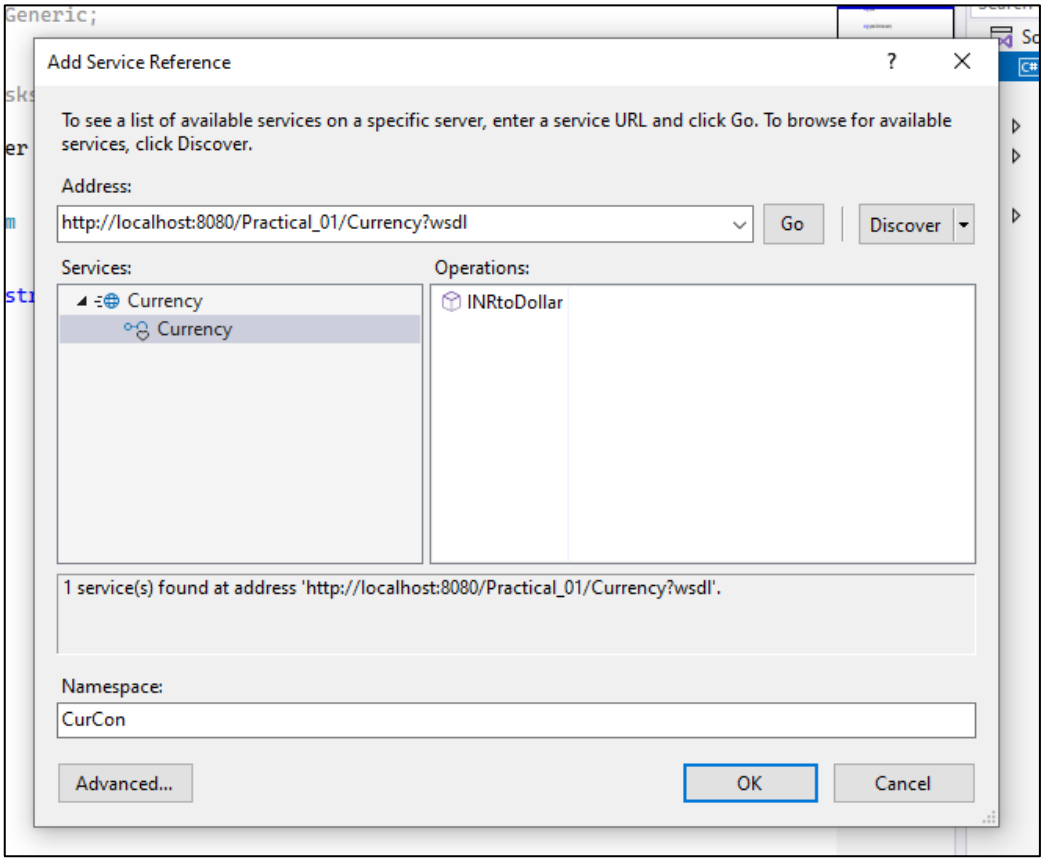
- CC Practical 1D –

NOTE: Make sure your Netbeans Web Service is already Deployed.

Step 3: Right Click your Project in 'Solution Explorer' on Right Side of the interface. Click on: **Add → Service Reference**



Step 4: On the dialog Box, Type the address of your Web Service then Click on Go. Also name your Namespace and proceed.



Step 5: Make Changes in Code.

```
Program.cs
CurrencyConverter | CurrencyConverter.Program | Main(string[] args)
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using CurrencyConverter.CurCon;

namespace CurrencyConverter
{
    0 references
    internal class Program
    {
        0 references
        static void Main(string[] args)
        {
            convClient client = new convClient();
            Console.WriteLine("Enter the Currency in INR: ");
            double d = double.Parse(Console.ReadLine());
            Console.WriteLine("\nSoln: The Indian Rupees " + d + " in Dollars is " + client.INRtoDollar(d));
            Console.WriteLine("\nEnter any key to Exit...");
            Console.ReadKey();
            Console.ReadLine();
        }
    }

    2 references
    internal class convClient
    {
        1 reference
        internal double INRtoDollar(double d)
        {
            return d / 85.11;
        }
    }
}
100 %   No issues found   Ln: 14   Ch: 50
```

Final Output

```
C:\Users\Admin\source\repos\CurrencyConverter\CurrencyConverter\bin\Debug\CurrencyConverter.exe
Enter the Currency in INR:
2568.87

Soln: The Indian Rupees 2568.87 in Dollars is 30.1829397250617

Enter any key to Exit...
```

- CC Practical 2A –

Aim: *Define a Simple SOAP Service like Temperature Conversion and Call it from different Platforms.*

STEPS:

Step 1: Create New Project and then Create Web Service to Convert Temperature (F to C).

Step 2: In the auto-generated Code after creating Web Service, do the following Changes.

```java
package server;

import javax.jws.WebService;
import javax.jws.WebMethod;
import javax.jws.WebParam;

/**
 *
 * @author Admin
 */
@WebService(serviceName = "Temp_Conversion")
public class Temp_Conversion {

    @WebMethod(operationName = "F_to_C")
    public String F_to_C(@WebParam(name = "fahrenheit") double fahrenheit) {
        //TODO write your implementation code here:
        return "The Fahrenhiet Temperature " + fahrenheit + "° in Celsius is " + ((fahrenheit-32) * 5/9) + "°";
    }
}
```

Step 3: Deploy Project and Test Web Service.

Output prompting Input to Convert Temperature (F to C).

Temp_Conversion Web Service Tester

This form will allow you to test your web service implementation (WSDL File)

To invoke an operation, fill the method parameter(s) input boxes and click on the button labeled with the method name.

Methods :

public abstract java.lang.String server.TempConversion.fToC(double)

fToC (50)

Final Output

fToC Method invocation

Method parameter(s)

Type	Value
double	50

Method returned

java.lang.String : "**The Fahrenhiet Temperature 50.0° in Celsius is 10.0°**"

- CC Practical 2B –

Aim: *Creating JSP Client*

STEPS:

Step 1: Create JSP (Jakarta Server Page).

Step 2: Write following Code in “Input” File.

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>JSP Page</title>
    </head>
    <body>
        <form action="Output.jsp">
            <pre>
                Enter the Temperature in Fahrenheit: <input type="text" name="t1" required>

                Enter the Temperature in Celsius: <input type="text" name="t2" required>
                <input type="submit"> <input type="reset">
            </pre>
        </form>
    </body>
</html>
```

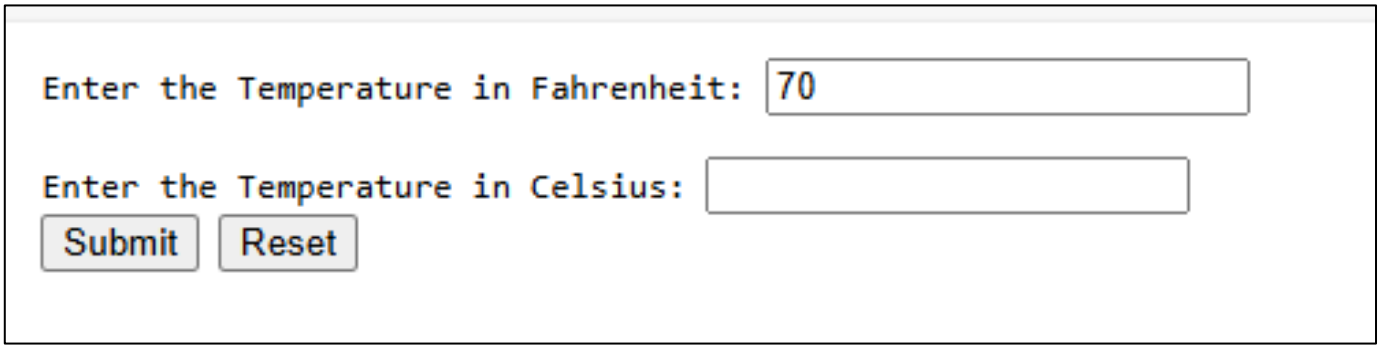
Step 3: Create Web Service Client for the Project. Specify Project URL and Type Package as “Client”.

Step 4: Create second JSP, name it as Output and Drag the Operation from **Web Service Refrences → Temp_Conversion → Temp_ConversionPort → F_to_C** into body tag.

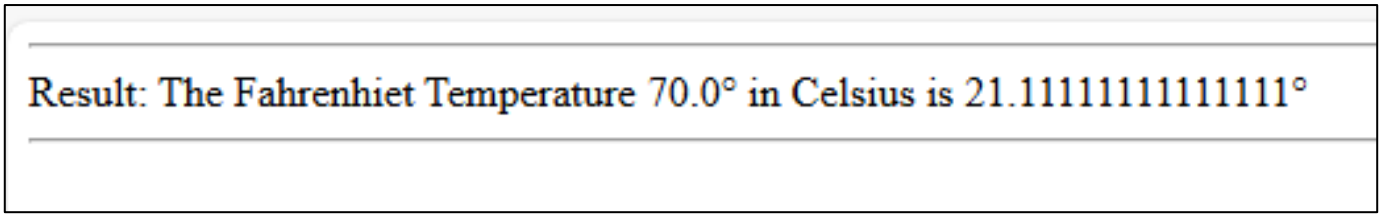
```
<%--
    Document   : Output
    Created on : 5 Jan, 2025, 12:57:43 PM
    Author     : Admin
--%>

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
    <head>
        <metag http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>JSP Page</title>
    </head>
    <body>

    <%-- start web service invocation --%><hr/>
    <%
    try {
        Client.TempConversion_Service service = new Client.TempConversion_Service();
        Client.TempConversion port = service.getTempConversionPort();
         // TODO initialize WS operation arguments here
        double fahrenheit = Double.parseDouble(request.getParameter("t1"));
        // TODO process result here
        java.lang.String result = port.fToC(fahrenheit);
        out.println("Result:\n "+result);
    } catch (Exception ex) {
        // TODO handle custom exceptions here
    }
    %>
    <%-- end web service invocation --%><hr/>

    </body>
</html>
```

Step 5: Deploy Project and Run the “Input” file.

Output prompting Input to Convert Temperature (F to C).



Final Output



- CC Practical 2C –

Aim: *Creating Python Client*

STEPS:

Step 1: Install Zeep Module and Implement the Code.

py_client.py - C:/Users/Admin/Desktop/Sem VI/CC/Practical_02/py_client.py (3.11.5)

File Edit Format Run Options Window Help

```python
from zeep import Client

wsdl_url = 'http://localhost:8080/Practical_02/Temp_Conversion?wsdl'
client = Client(wsdl_url)

print("Temperature Converter:\n")
f = float(input("Enter Temperature in Fahrenheit: "))
result = client.service.F_to_C(f)
print(result)
```

Final Output

```
= RESTART: C:/Users/Admin/Desktop/Sem VI/CC/Practical_02/py_client.py
Temperature Converter:

Enter Temperature in Fahrenheit: 90
The Fahrenhiet Temperature 90.0° in Celsius is 32.22222222222222°
```

- CC Practical 2D –

Aim: *Creating .NET Client*

STEPS:

Step 1: Open Visual Studio and Create a New Project. Search for “Console C#” then select “Console App” for .NET freamwork.

Step 2: Name the Project.

NOTE: Make sure your Netbeans Web Service is already Deployed.

Step 3: Right Click your Project in ‘Solution Explorer’ on Right Side of the interface. Click on: **Add → Service Reference**

Step 4: On the dialog Box, Type the address of your Web Service then Click on Go. Also name your Namespace and proceed.

Step 5: Make Changes in Code.

```
TempConverter | TempConverter.Program | Main(string[] args)

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using TempConverter.TempConv;

namespace TempConverter
{
    0 references
    internal class Program
    {
        0 references
        static void Main(string[] args)
        {
            convClient client = new convClient();
            Console.WriteLine("Enter the Temperature in Fahrenheit: ");
            double f = double.Parse(Console.ReadLine());
            Console.WriteLine("\nSoln: The Temperature in Fahrenheit " + f + "° in Celsius is " + client.F_to_C(f) + "°");
            Console.WriteLine("\nEnter any key to Exit...");
            Console.ReadKey();
            Console.ReadLine();
        }
    }

    2 references
    internal class convClient
    {
        1 reference
        internal double F_to_C(double f)
        {
            return ((f - 32) * 5/9);
        }
    }
}
```

Final Output

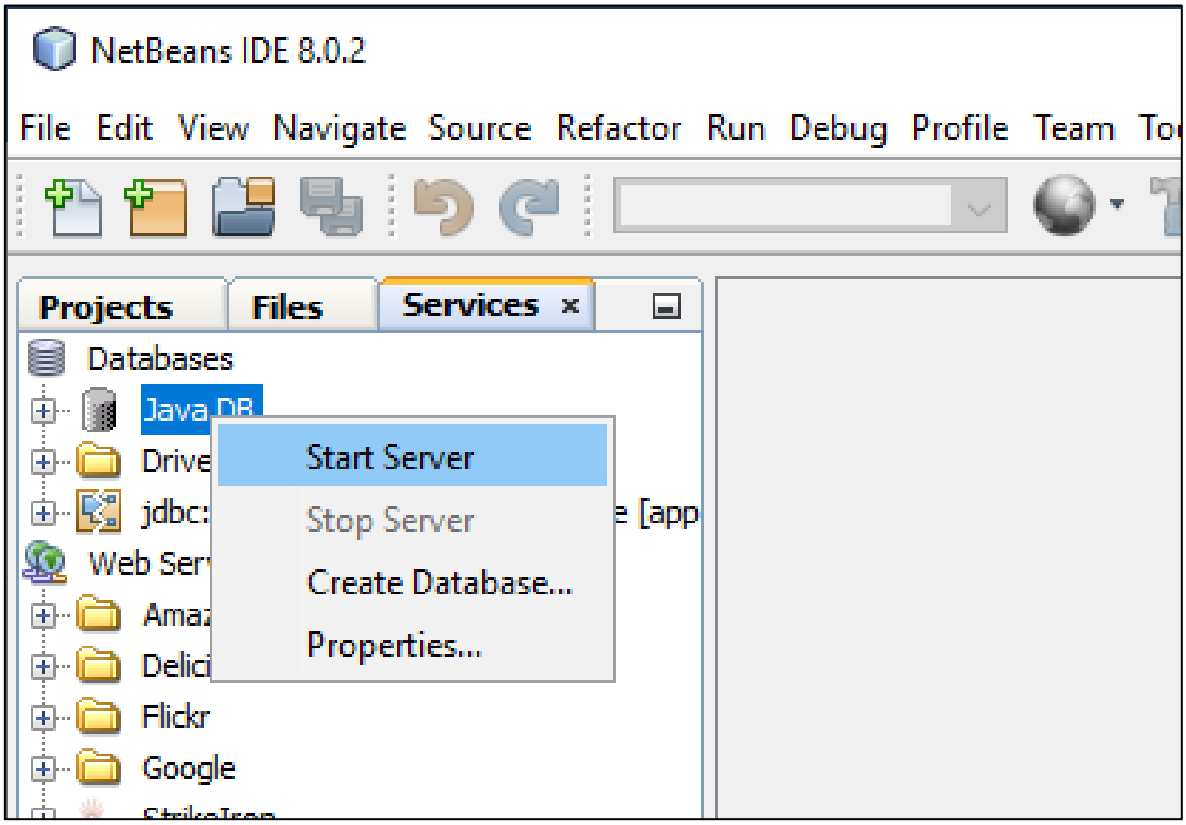
```
C:\Users\Admin\source\repos\TempConverter\TempConverter\bin\Debug\TempConverter.exe
Enter the Temperature in Fahrenheit:
121

Soln: The Temperature in Fahrenheit 121° in Celsius is 49.4444444444444°

Enter any key to Exit...
```

- CC Practical 03 –

Aim: *Create a Simple REST Service.*

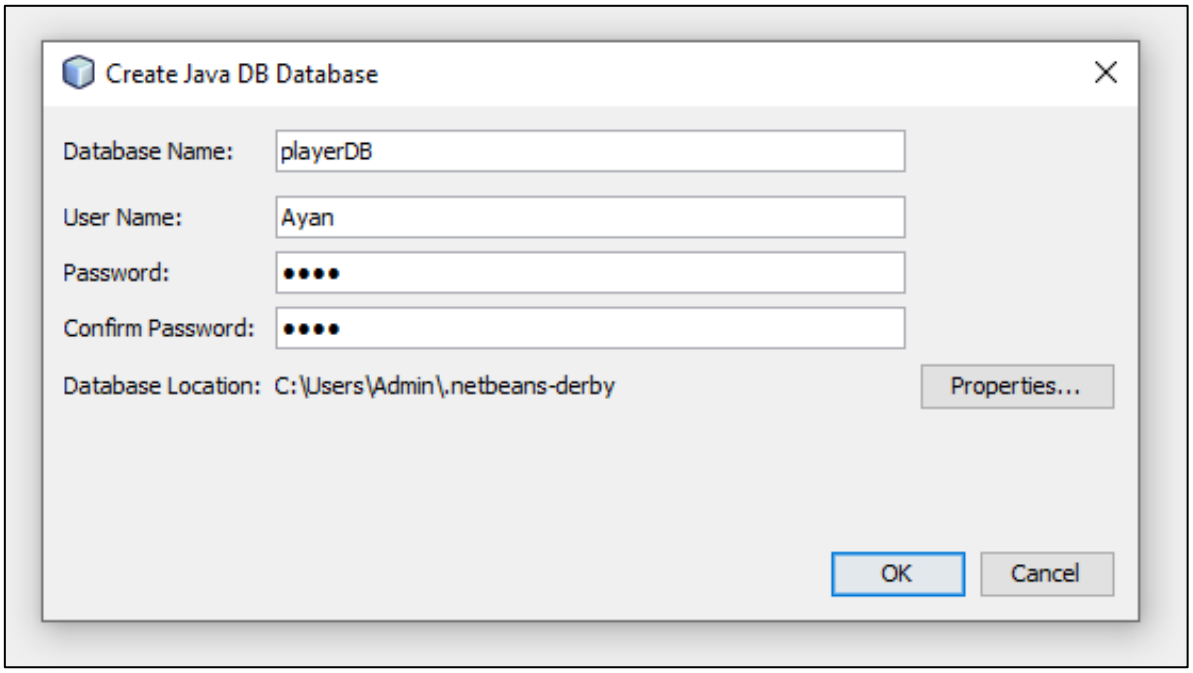
STEPS:

Step 1: Start Java Database by performing following steps, **Services Tab → Expand Database Node → Right Click Java DB Icon → Select Start Server.**

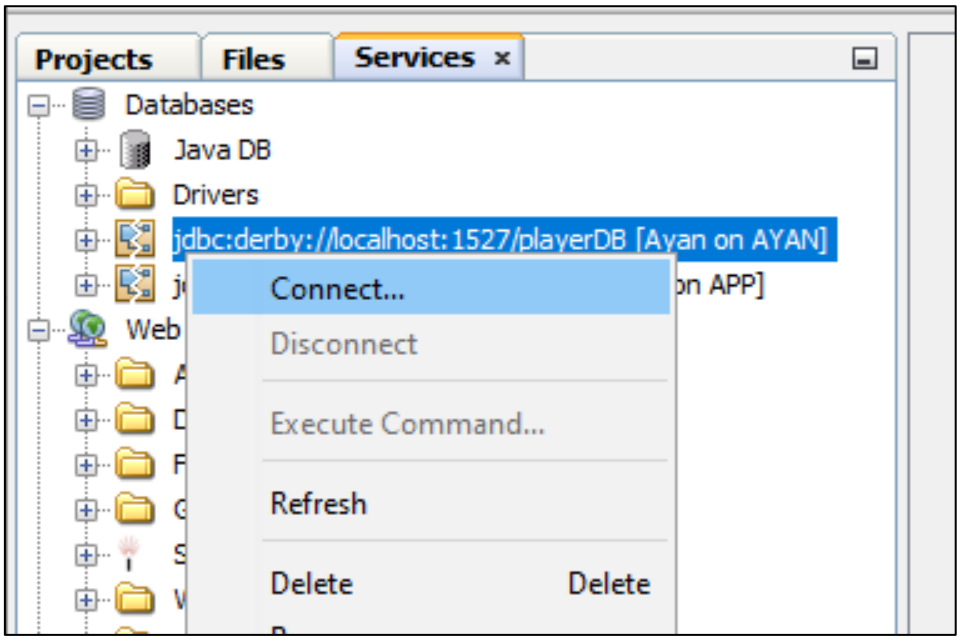


Step 2: Again, Right Click on Java DB and Click on 'Create Database'.

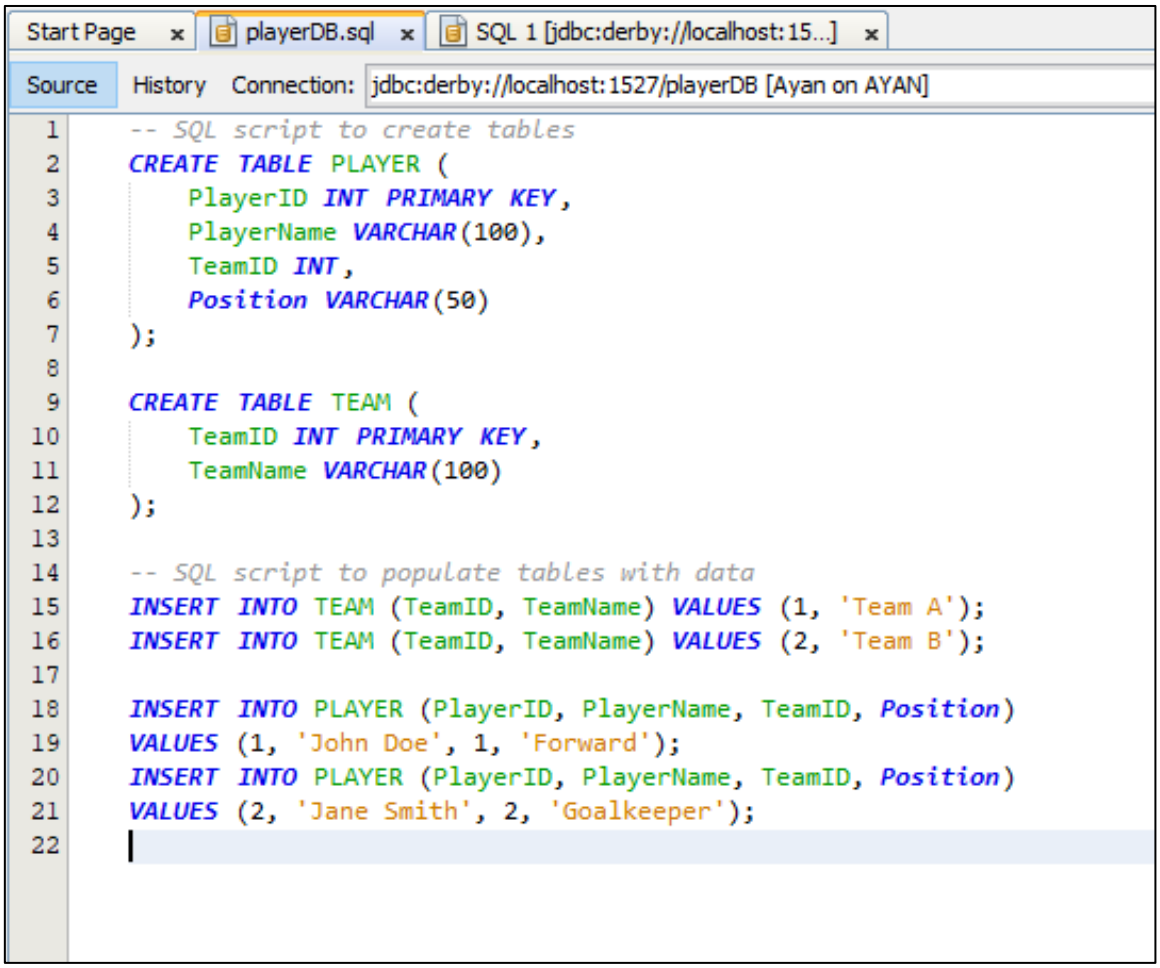
Step 3: Fill in the details. (DB Name, Username, Password, Confirm Pwd)



Step 4: Establish Connection with DB by right clicking on jdbc:derby://localhost:1527/playerDB, then select Connect.

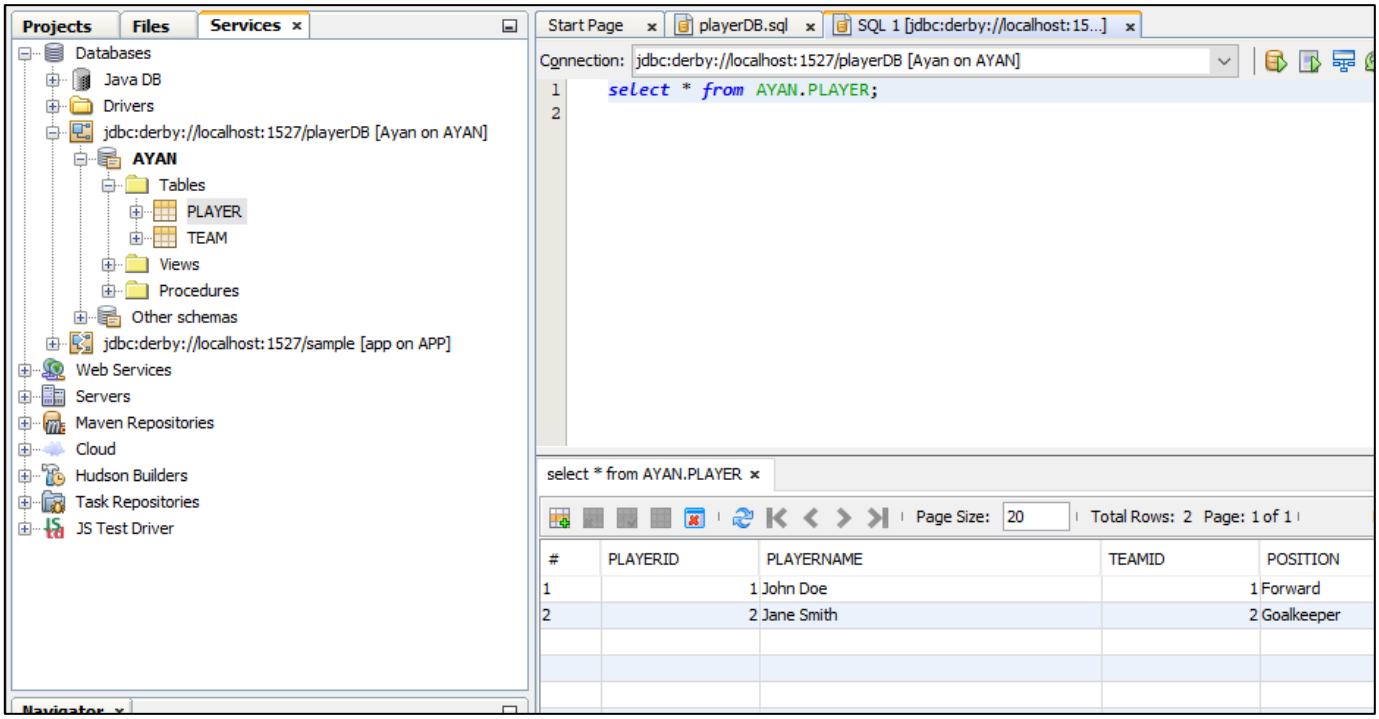


Step 5: Create playerDB.sql file and Write down some Queries.

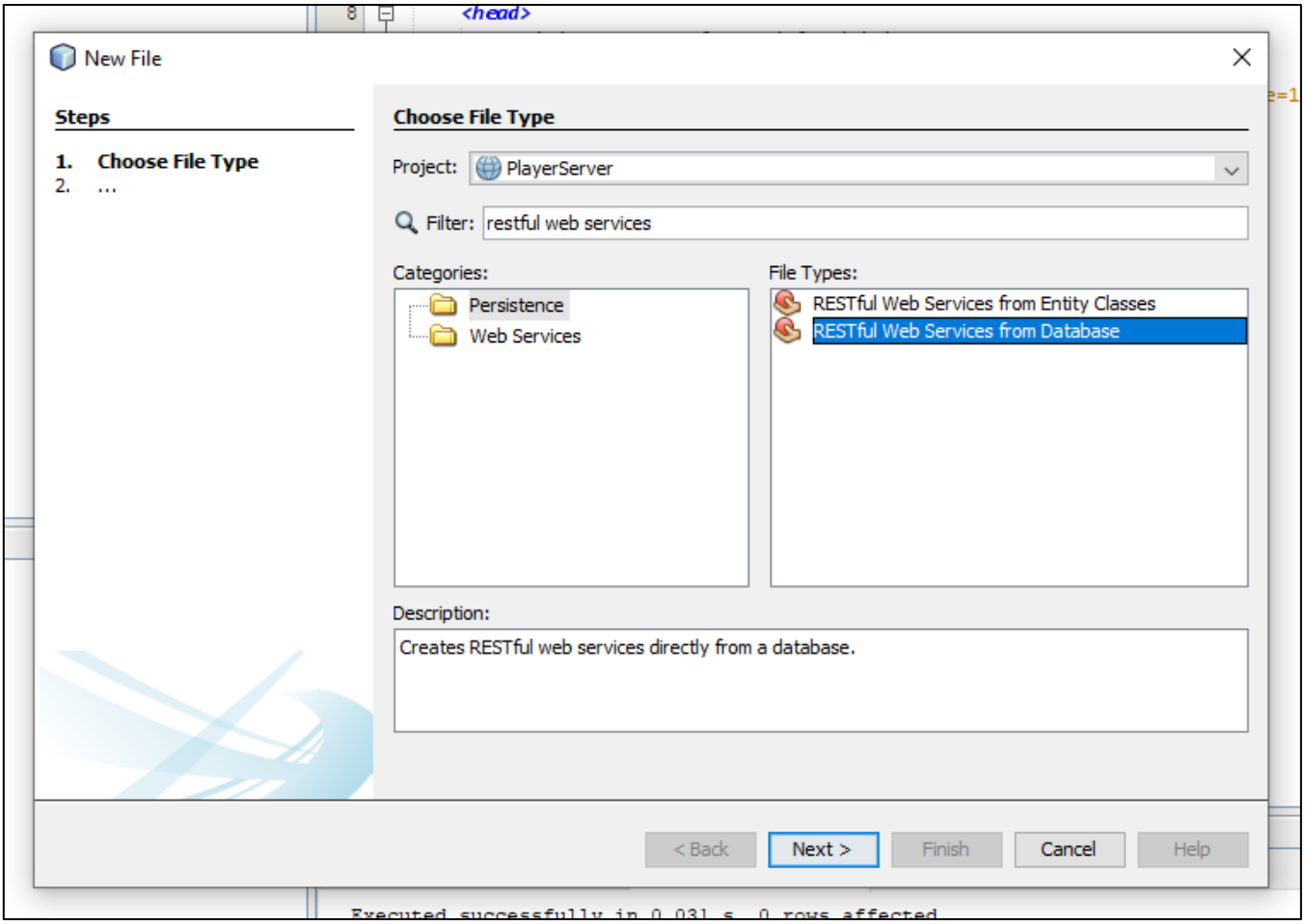



```sql
-- SQL script to create tables
CREATE TABLE PLAYER (
    PlayerID INT PRIMARY KEY,
    PlayerName VARCHAR(100),
    TeamID INT,
    Position VARCHAR(50)
);

CREATE TABLE TEAM (
    TeamID INT PRIMARY KEY,
    TeamName VARCHAR(100)
);

-- SQL script to populate tables with data
INSERT INTO TEAM (TeamID, TeamName) VALUES (1, 'Team A');
INSERT INTO TEAM (TeamID, TeamName) VALUES (2, 'Team B');

INSERT INTO PLAYER (PlayerID, PlayerName, TeamID, Position)
VALUES (1, 'John Doe', 1, 'Forward');
INSERT INTO PLAYER (PlayerID, PlayerName, TeamID, Position)
VALUES (2, 'Jane Smith', 2, 'Goalkeeper');
```

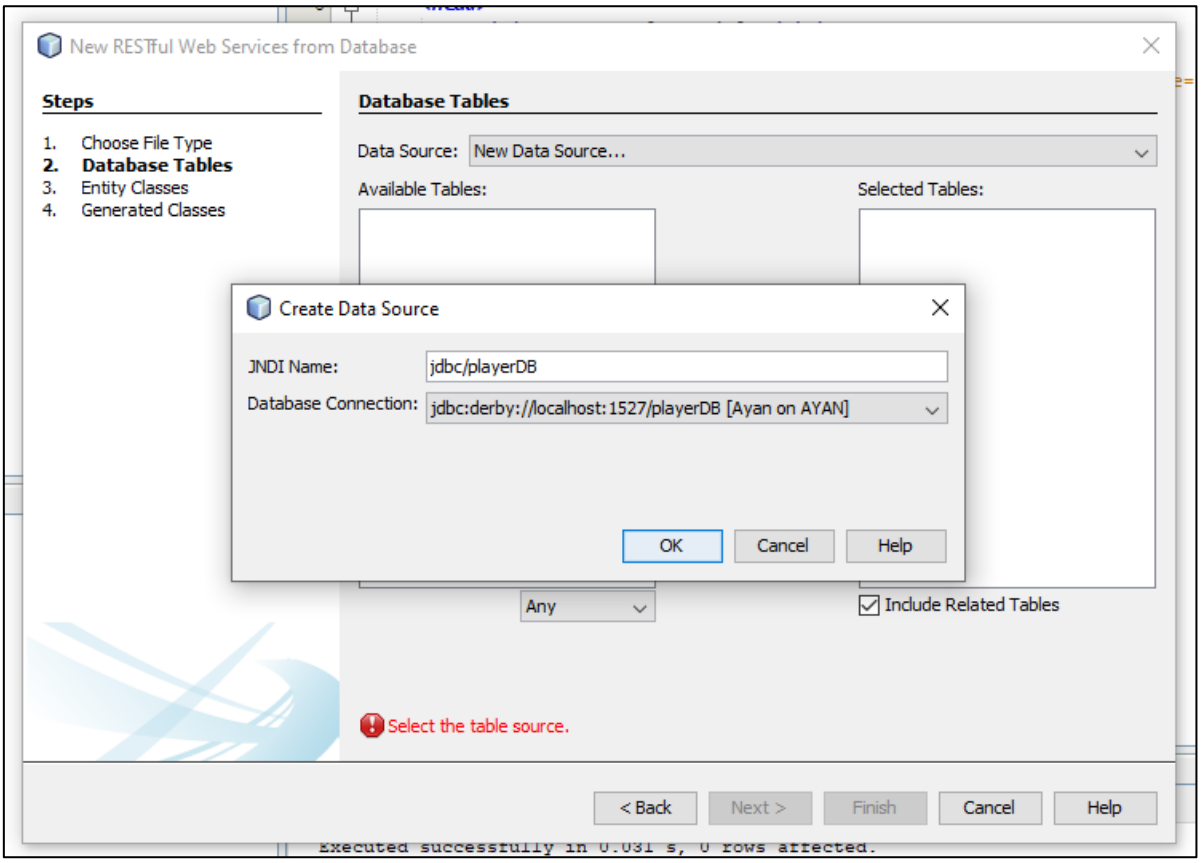
Step 6: View and examine your data



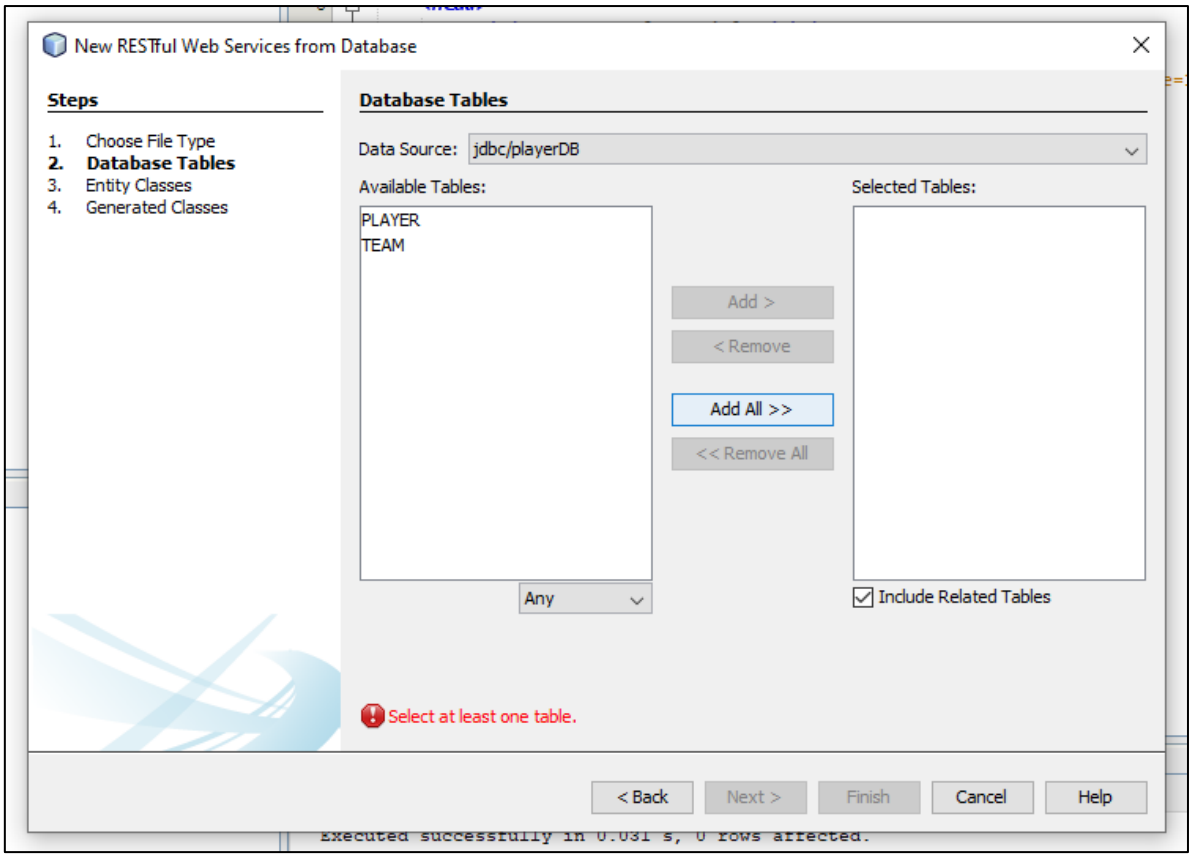
Step 7: Now create a new project (**Java Web → Web App**) named PlayerServer, then create a 'RESTful web service from Database' (Search in Filter).



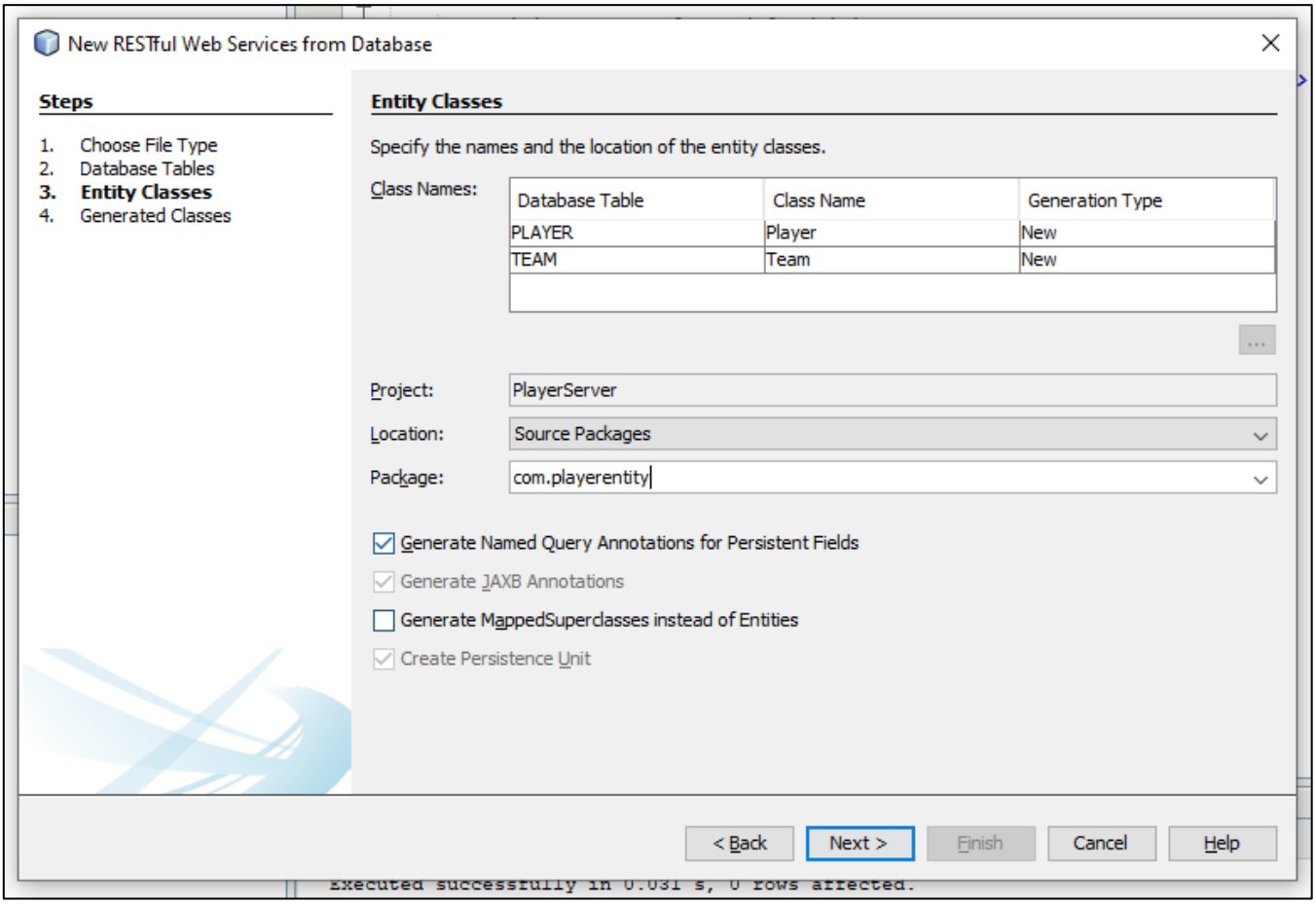
Step 8: Select data source, in the drop down menu select New data source. In create Data source dialog box, type JNDI Name and select DB Connection. Click OK



Step 9: Next Select Add All to add all the tables and Click Next

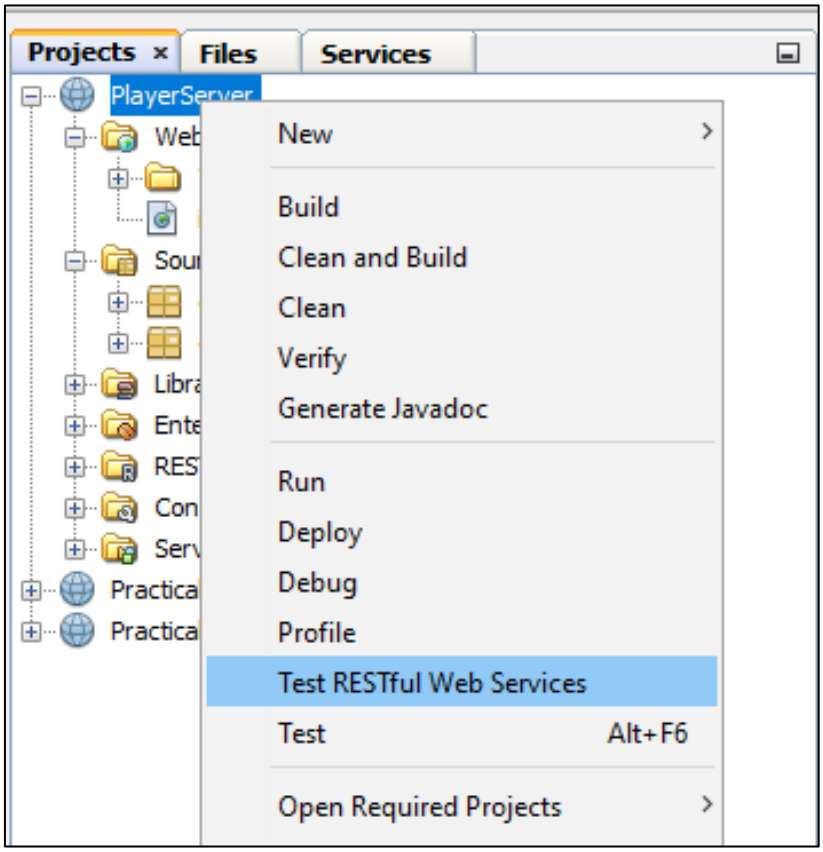


Step 10: In Entity Classes, name Package. Click Next

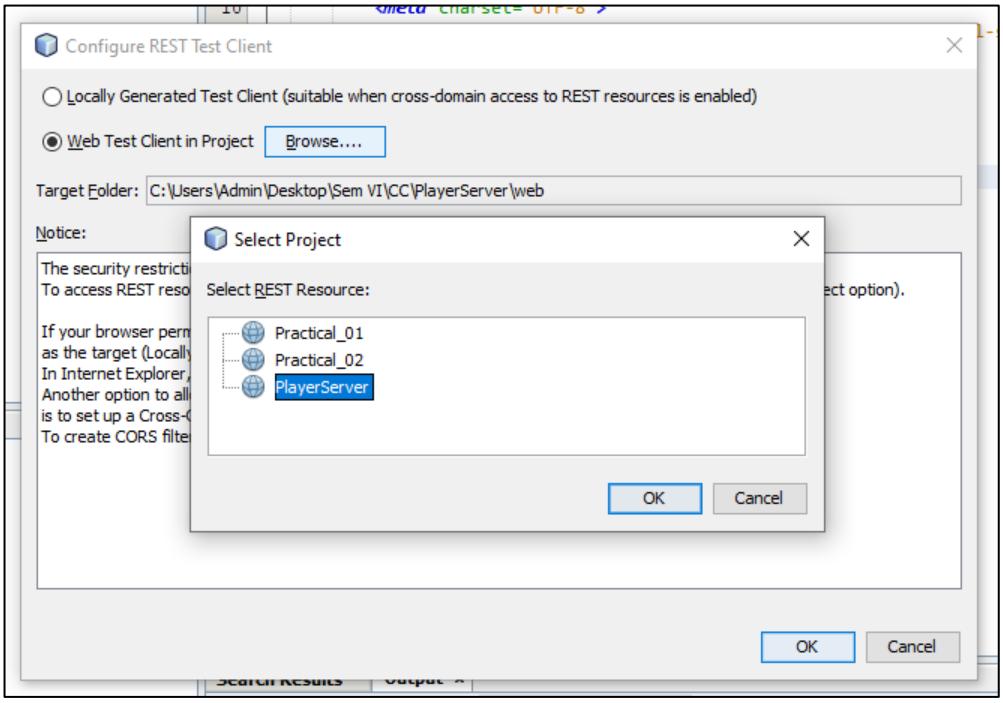


Step 11: Now in Generated Classes Window, let the default values be and Click Finish.

Step 12: Right Click your project and Select Test RESTful Web Services.

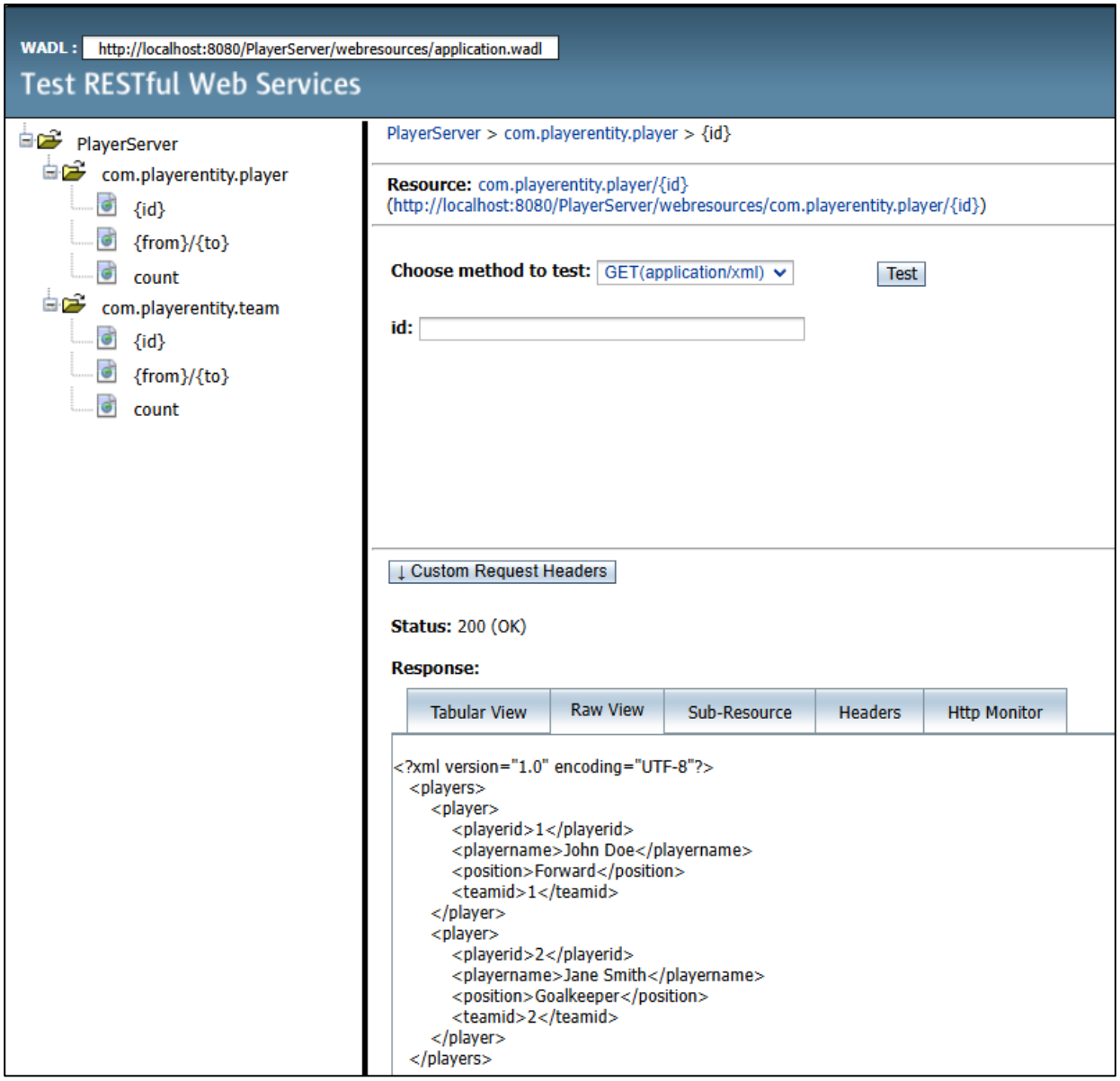


Step 13: In below Window, Browse Test Client then select PlayerServer project and proceed.



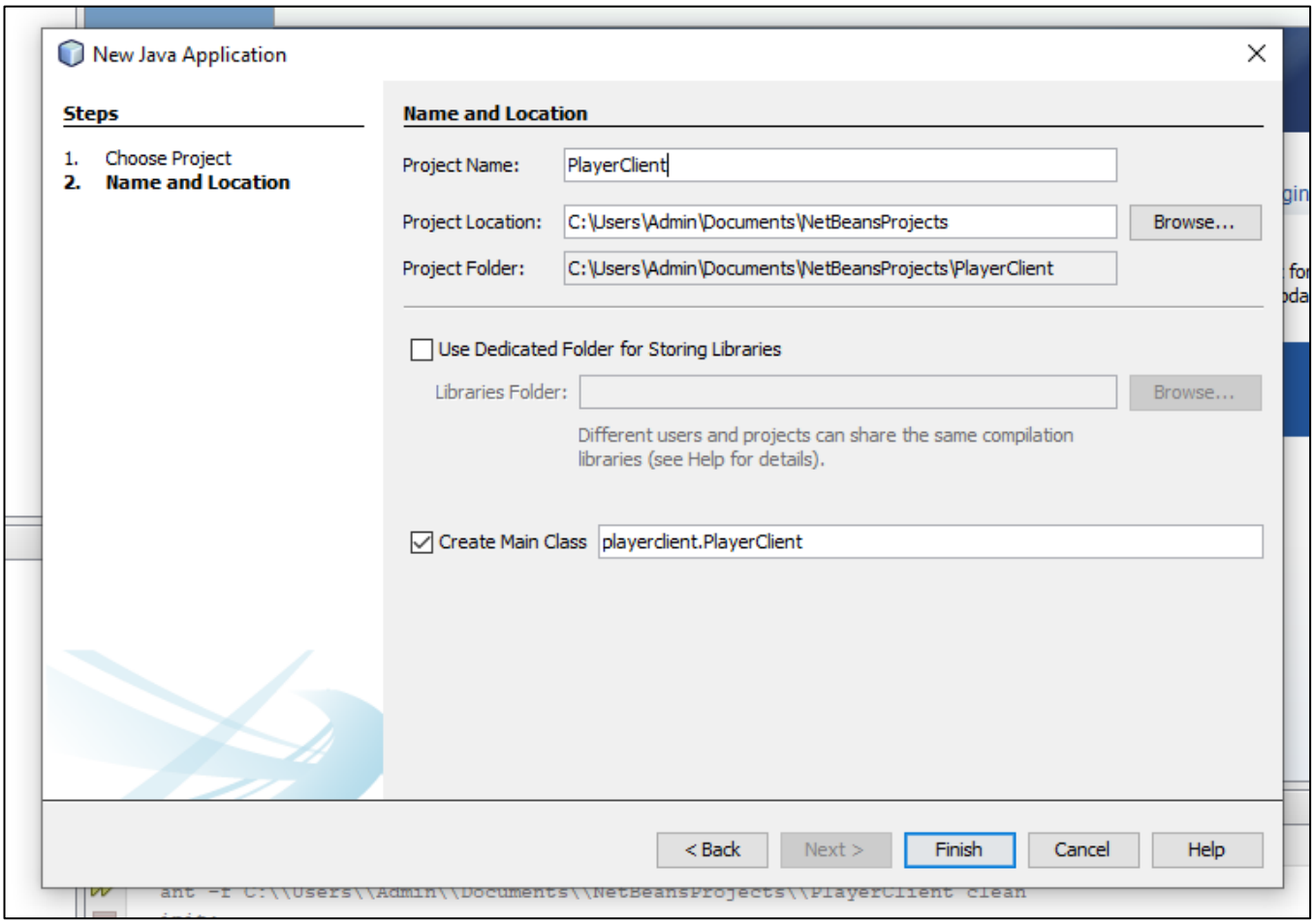
OUTPUT:

- Choose **GET(application/xml)** Method to test And Click on Test.
- Here, Response is viewed in Raw View.

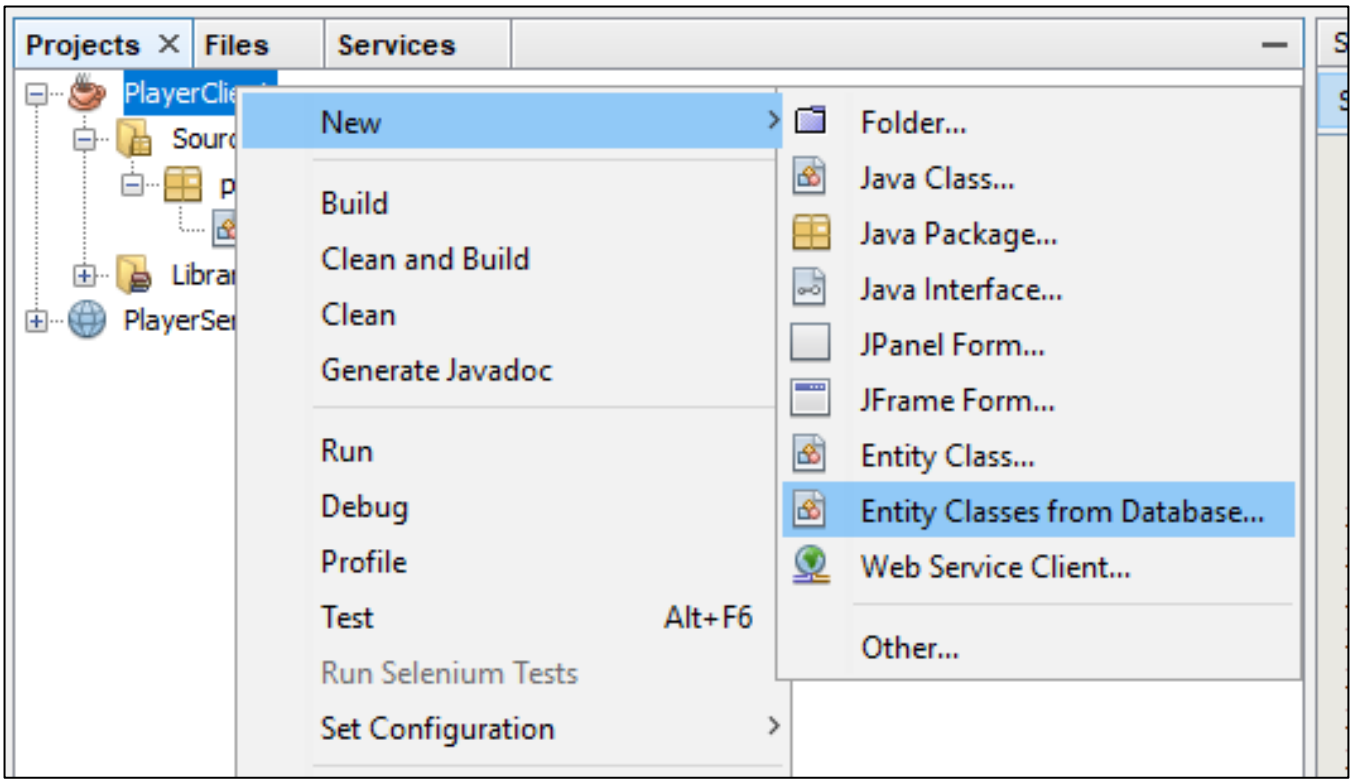


Implementing CRUD Operations: It needs creation of Entity Classes to communicate with DB.

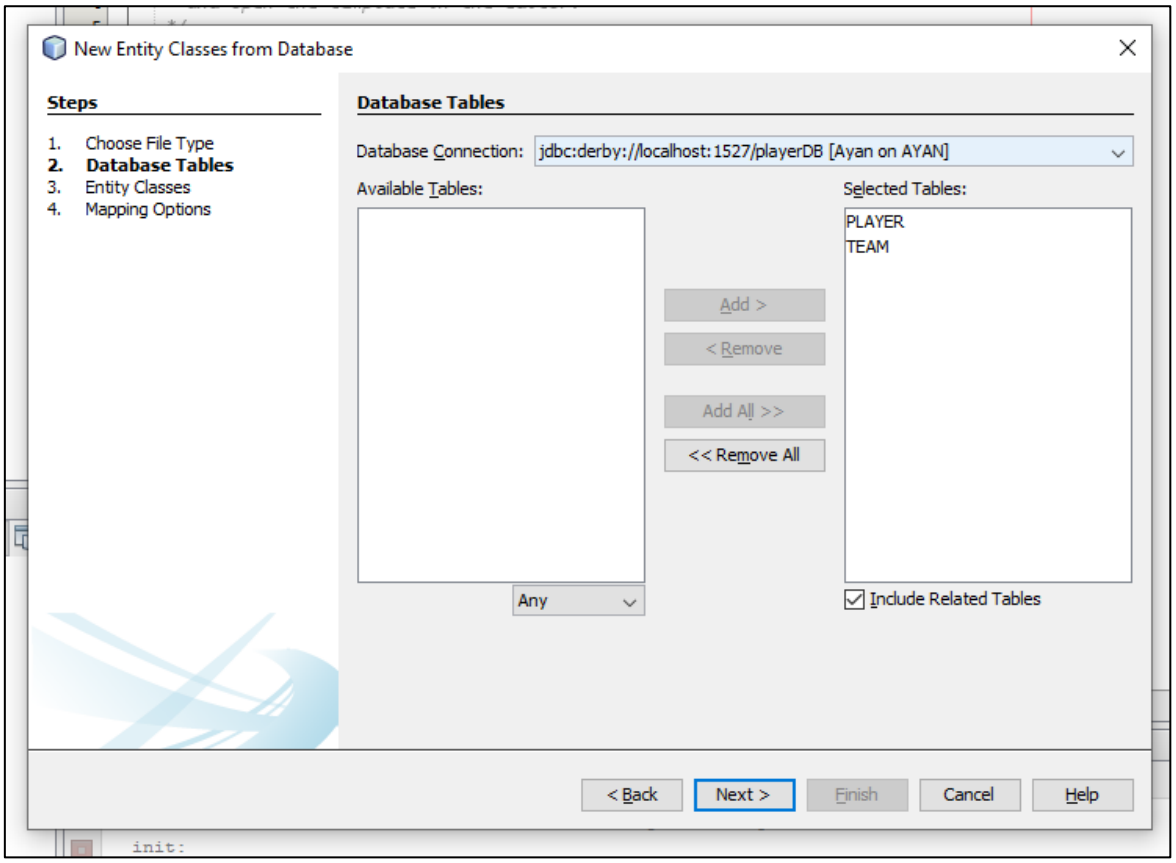
Step 14: Create a new Project (**Java → Java App**) named PlayerClient.



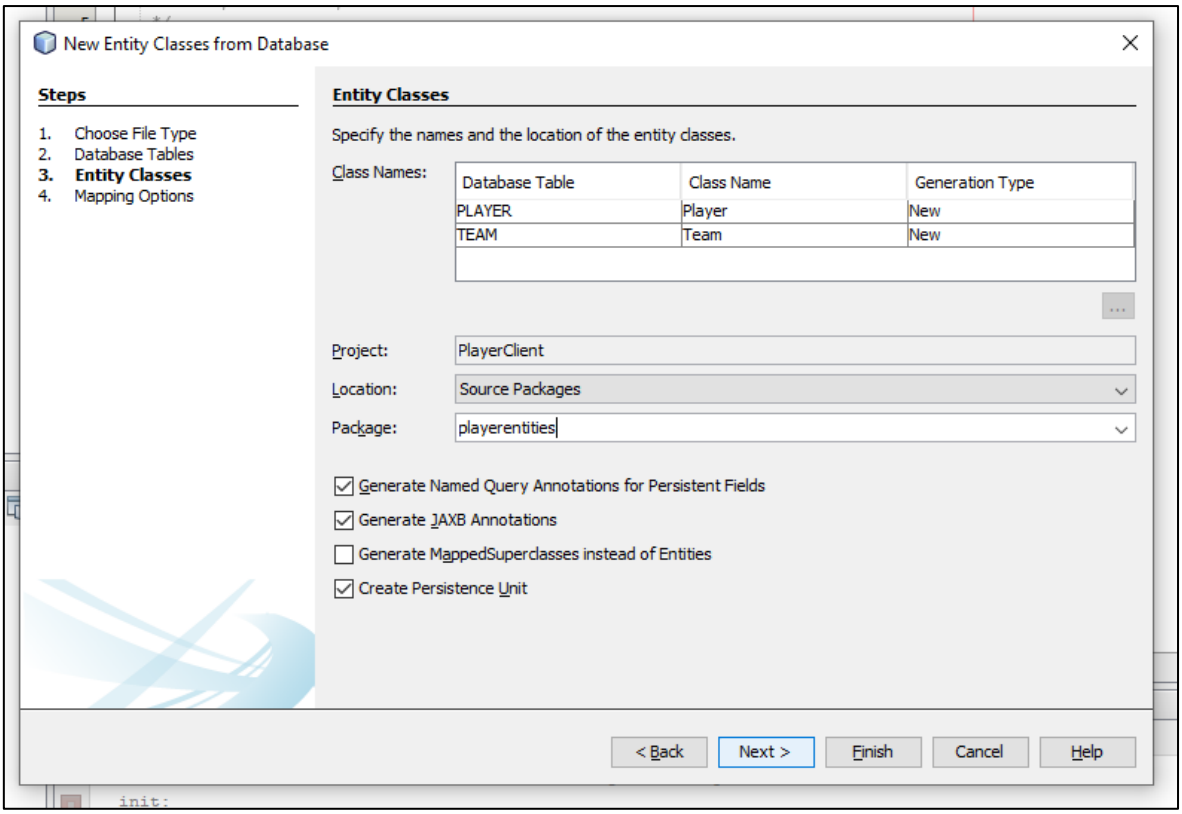
Step 15: Right Click on Project, Select **New → Entity Classes from Database.**



Step 16: Now, Choose you DB Connection from Database Connection drop down Menu and then Select Add All. Click Next.

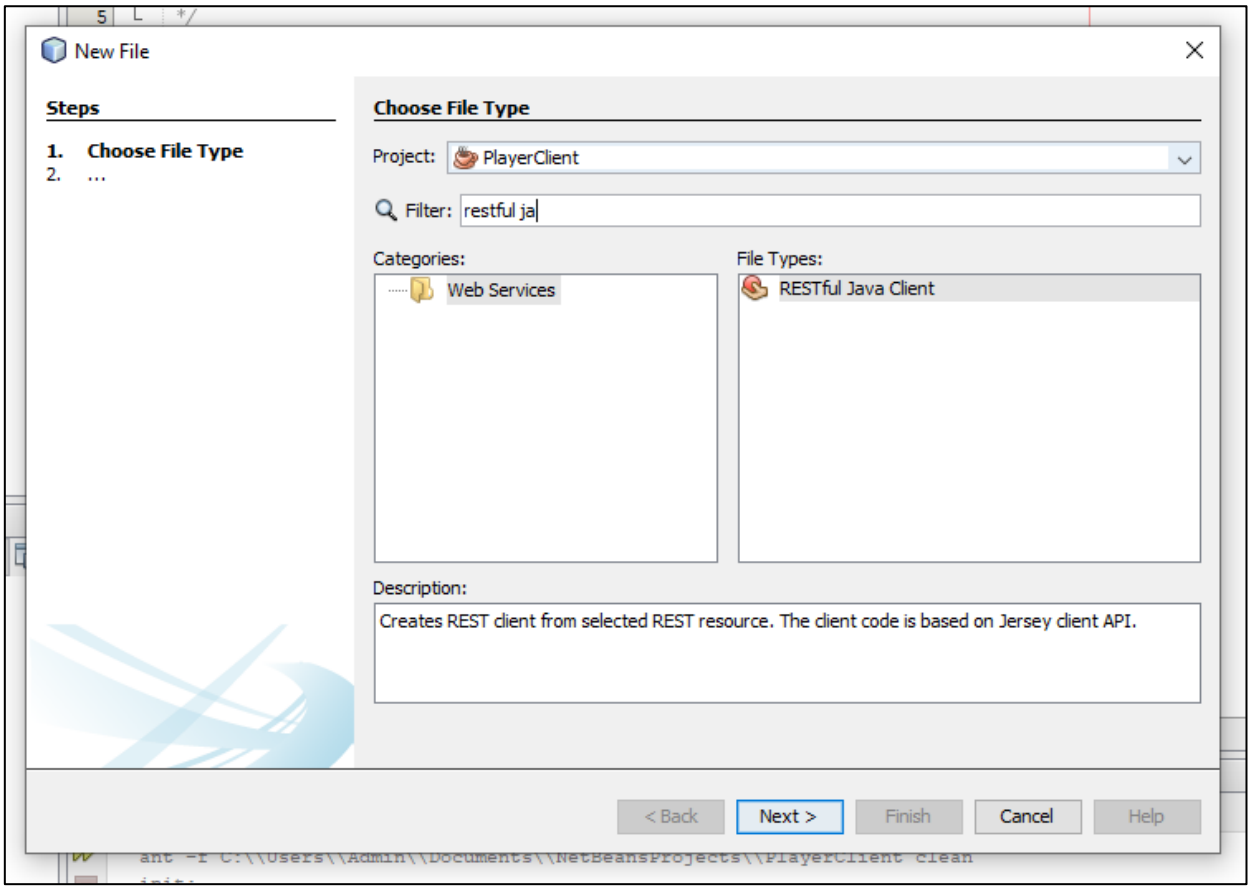


Step 17: Name you package as playerentities.



Step 18: Let default vals be in Mapping Options and Click Finish.

Step 19: Now Create Client by Right Clicking on PlayerClient Project. Choose **Others → RESTful Java Client.** (Search in Filter)

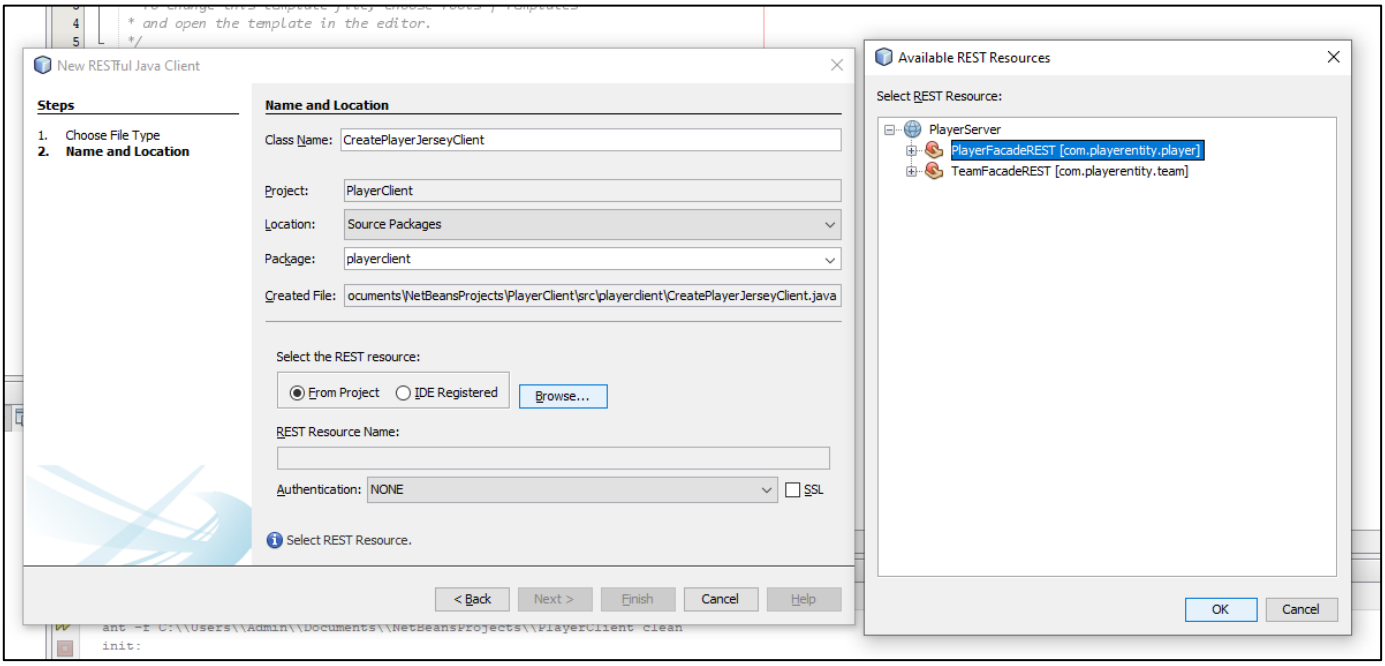


NOTE: *Better to Name Client as **"CRUDPlayerClient"** as this single Client can be used to perform all 4 Operations rather than creating 4 different Clients for each Operation.*

Step 20: i. Name your Client Name as CreatePlayerJerseyClient.

ii. Select playerclient package from drop down menu.

iii. Select REST resource by Browsing Project. Expand PlayerServer Project, Choose PlayerFacadeREST. Click OK and then Click Finish.



Step 21: Start Writing Foll Code in PlayerClient.java file's Main Method **to Insert New Player in DB**.

```java
package playerclient;

import javax.ws.rs.core.Response;
import javax.ws.rs.core.GenericType;
import java.util.List;
import playerentities.Player;

public class PlayerClient {
    public static void main(String[] args) {

        CreatePlayerJerseyClient client1 = new CreatePlayerJerseyClient();

        /* ClientResponse response = client1.findAll_XML(ClientResponse.class);
        GenericType<List<Player>>genericType = new GenericType<List<Player>>(){};
        List<Player> data = new ArrayList<>();
        data = response.getEntity(genericType); */

        Response response = client1.findAll_XML(Response.class);
        List<Player> data = response.readEntity(new GenericType<List<Player>>() {});

        // Create a new player
        Player p = new Player();
        p.setPlayerid(5);
        p.setPlayername("Tom");
        p.setTeamid(11);
        p.setPosition("Defender");

        client1.create_XML(p);

        System.out.println("New player created successfully!");
    }
}
```

NOTE: *Earlier Commented Code supports older version of **JERSEY 1.x**, a JAR File Library which is used for handling responses and building RESTful APIs. Now we used new Version **JAX-RS 2.x**, hence slightly changed Code can be observed.*

Step 22: Make sure PlayerServer is Deployed before Running above Code.

Output:

```
run:
New player created successfully!
BUILD SUCCESSFUL (total time: 0 seconds)
```

- CC Practical 03 –

DB Earlier:

SELECT * FROM AYAN.PLAYER... ×

Max. rows: 100 | Fetched Rows: 2 | Matching Rows:

#	PLAYERID	PLAYERNAME	TEAMID	POSITION
1	1	John Doe	1	Forward
2	2	Jane Smith	2	Goalkeeper

DB Now:

SELECT * FROM AYAN.PLAYER... ×

Max. rows: 100 | Fetched Rows: 3 | Matching Rows:

#	PLAYERID	PLAYERNAME	TEAMID	POSITION
1	1	John Doe	1	Forward
2	2	Jane Smith	2	Goalkeeper
3	5	Tom	11	Defender

Step 23: Implement foll Code by either Creating a new class (**Right Click on playerclient → New → Java Class → Name it**) or Replacing previous Code **to Read Data from DB.**

Source | History

```java
/*
 * To change this license header, choose License Headers in Project Properties.
 * To change this template file, choose Tools | Templates
 * and open the template in the editor.
 */
package playerclient;

import javax.ws.rs.core.Response;
import javax.ws.rs.core.GenericType;
import java.util.List;
import playerentities.Player;

/**
 *
 * @author Admin
 */
public class ReadClient {
    public static void main(String[] args) {

        CreatePlayerJerseyClient client1 = new CreatePlayerJerseyClient();
        Response response = client1.findAll_XML(Response.class);
        List<Player> data = response.readEntity(new GenericType<List<Player>>() {});

        System.out.println("Retreiving and Displaying Player Details:\n");

        for (Player players : data) {
            System.out.println("Player ID: " + players.getPlayerid());
            System.out.println("Player Name: " + players.getPlayername());
            System.out.println("Team ID: " + players.getTeamid());
            System.out.println("Position: " + players.getPosition() + "\n");
        }

        System.out.println("Data Retreived!");
    }
}
```

OUTPUT:

```
run:
Retreiving and Displaying Player Details:

Player ID: 1
Player Name: John Doe
Team ID: 1
Position: Forward

Player ID: 2
Player Name: Jane Smith
Team ID: 2
Position: Goalkeeper

Player ID: 5
Player Name: Tom
Team ID: 11
Position: Defender

Data Retreived!
BUILD SUCCESSFUL (total time: 0 seconds)
```

Step 24: Implement Foll Code **to Delete a Client (Player) from DB.**

```java
/*
 * To change this license header, choose License Headers in Project Properties.
 * To change this template file, choose Tools | Templates
 * and open the template in the editor.
 */
package playerclient;

import javax.ws.rs.core.Response;
import javax.ws.rs.core.GenericType;
import java.util.List;
import playerentities.Player;
/**
 *
 * @author Admin
 */
public class DelClient {

    public static void main(String[] args) {

        CreatePlayerJerseyClient client1 = new CreatePlayerJerseyClient();
        client1.remove("1");
    }

}
```

OUTPUT:

SELECT * FROM AYAN.PLAYER... ×

Max. rows: 100 | Fetched Rows: 2 | Matching R

#	PLAYERID	PLAYERNAME	TEAMID	POSITION
1	2	Jane Smith	2	Goalkeeper
2	5	Tom	11	Defender

Step 25: Implement Foll Code to Update a Client (Player) from DB.

```java
package playerclient;

import javax.ws.rs.core.Response;
import javax.ws.rs.core.GenericType;
import java.util.List;
import playerentities.Player;
/**
 *
 * @author Admin
 */
public class UpdateClient {
    public static void main(String[] args) {

        CreatePlayerJerseyClient client1 = new CreatePlayerJerseyClient();

        Response response = client1.find_XML(Response.class, "2");
        GenericType<Player> generictype = new GenericType<Player>(){};
        Player player = response.readEntity(generictype);

        System.out.println("Player ID: " + player.getPlayerid());
        System.out.println("Player Name: " + player.getPlayername());
        System.out.println("Team ID: " + player.getTeamid());
        System.out.println("Position: " + player.getPosition());

        player.setTeamid(21);
        player.setPosition("Forward");

        client1.edit_XML(player, "2");

        System.out.println("\nAfter Update:");
        System.out.println("Player ID: " + player.getPlayerid());
        System.out.println("Player Name: " + player.getPlayername());
        System.out.println("Team ID: " + player.getTeamid());
        System.out.println("Position: " + player.getPosition());
    }
}
```

- CC Practical 03 –

OUTPUT:

```
run:
Player ID: 2
Player Name: Jane Smith
Team ID: 2
Position: Goalkeeper

After Update:
Player ID: 2
Player Name: Jane Smith
Team ID: 21
Position: Forward
BUILD SUCCESSFUL (total time: 0 seconds)
```

DB:

SELECT * FROM AYAN.PLAYER...

Max. rows: 100 | Fetched Rows: 2

#	PLAYERID	PLAYERNAME	TEAMID	POSITION
1	2	Jane Smith	21	Forward
2	5	Tom	11	Defender

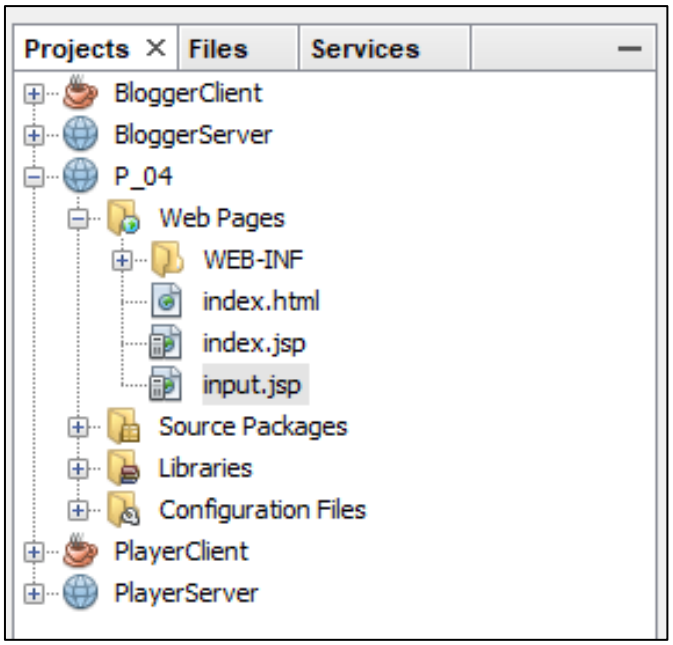
---- Enjoy your Practical 😄😃 ----

- CC Practical 04 –

Aim: *Develop application to consume Google's search / Google's Map RESTful Web service.*

STEPS:

Step 1: Create a Java Web application and create two jsp pages- input.jsp and index.jsp



Step 2: Write down below code in input.jsp file.

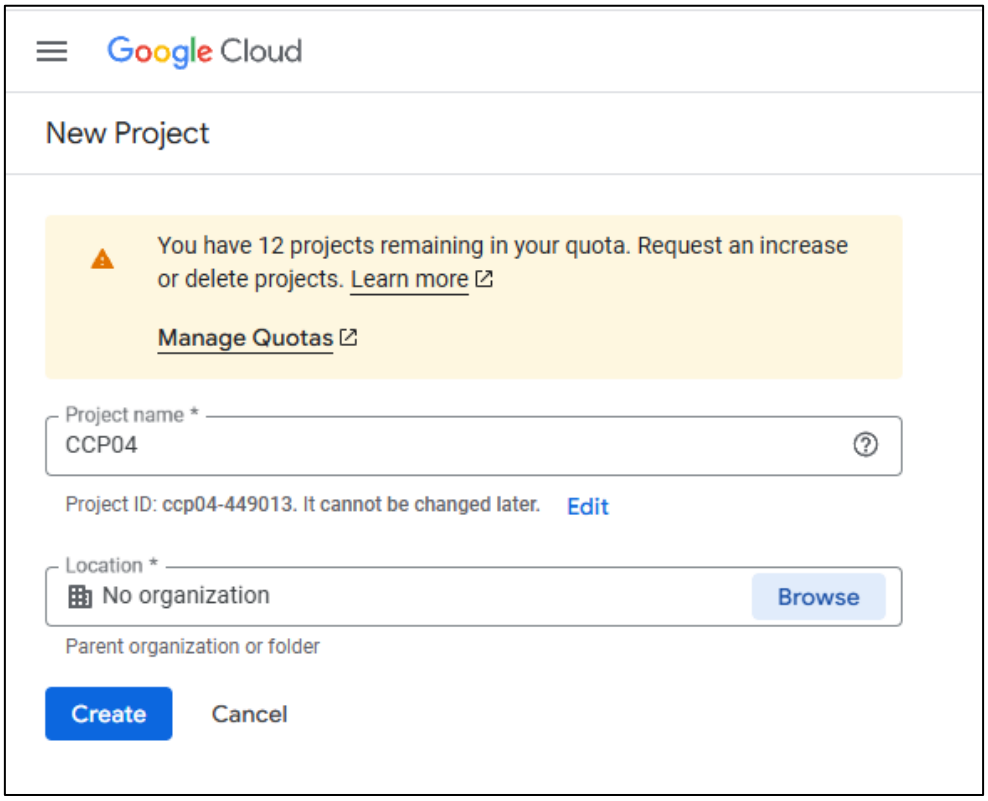
Start Page | index.jsp | input.jsp

Source | History

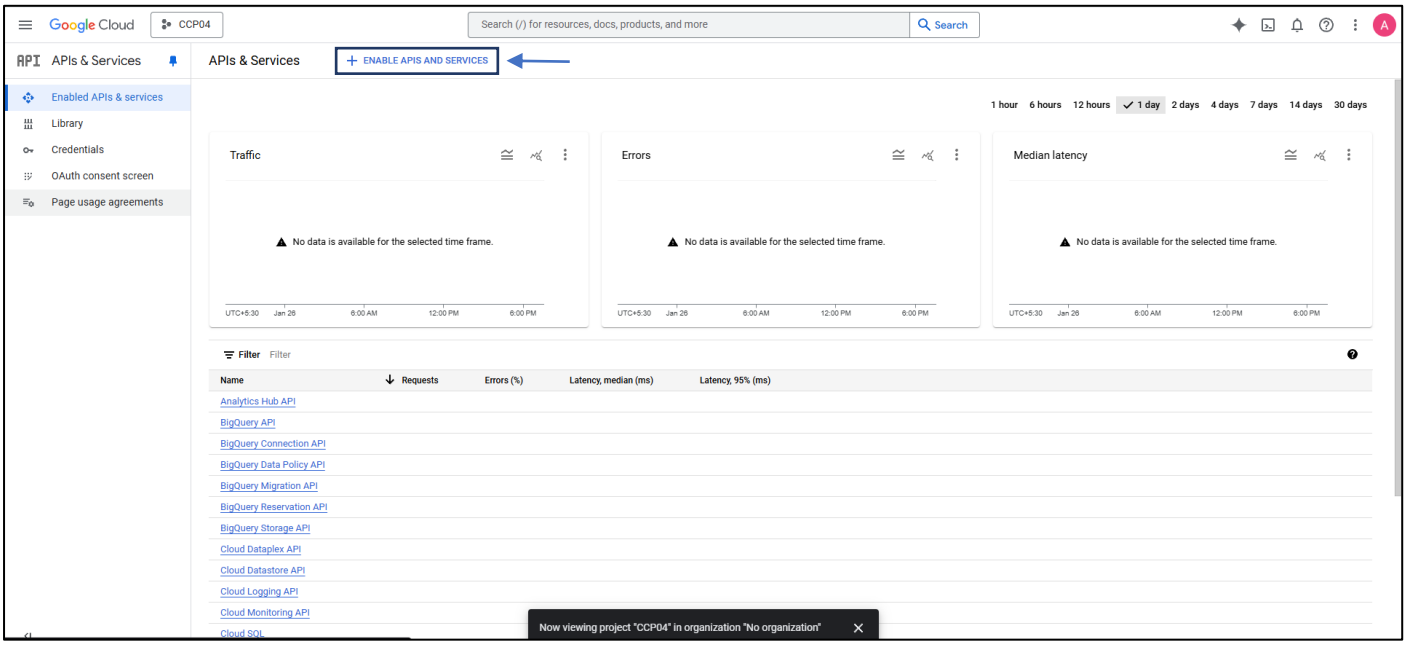
```jsp
<%--
    Document   : input
    Created on : 26 Jan, 2025, 4:07:06 PM
    Author     : Admin
--%>

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>JSP Page</title>
    </head>
    <body>
        <h1>Map API</h1>

        <form action="index.jsp">
            <pre>
                Enter Latitude: <input type="text" name="t1"/>
                Enter Longitude: <input type="text" name="t2"/>
                <input type="submit" value="Show"/>
            </pre>
        <form/>

    </body>
</html>
```

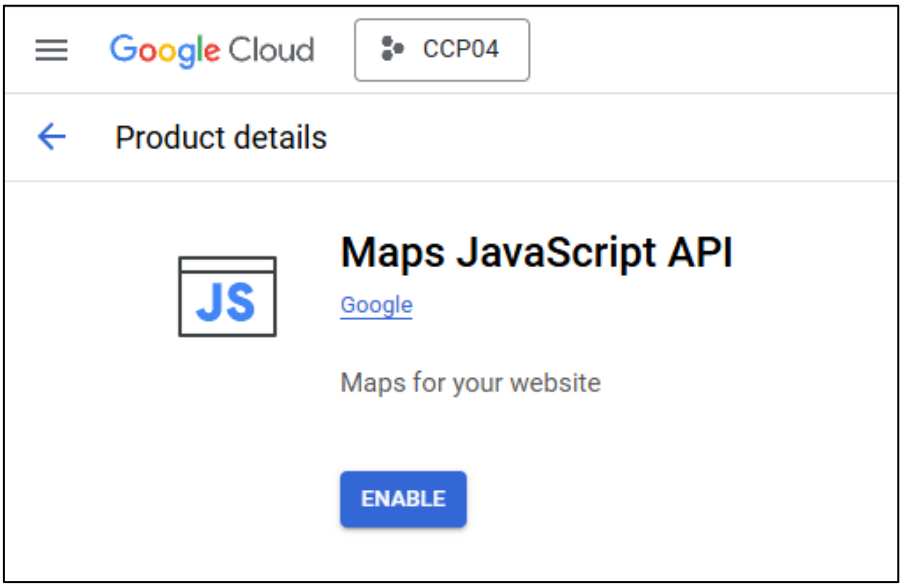
Step 3: Before running the application, we need the Google API key. The steps are shown here: **Visit Google APIs Console (https://console.developers.google.com, you have to login with your Google account)**

Step 4: Now create a project as shown below.

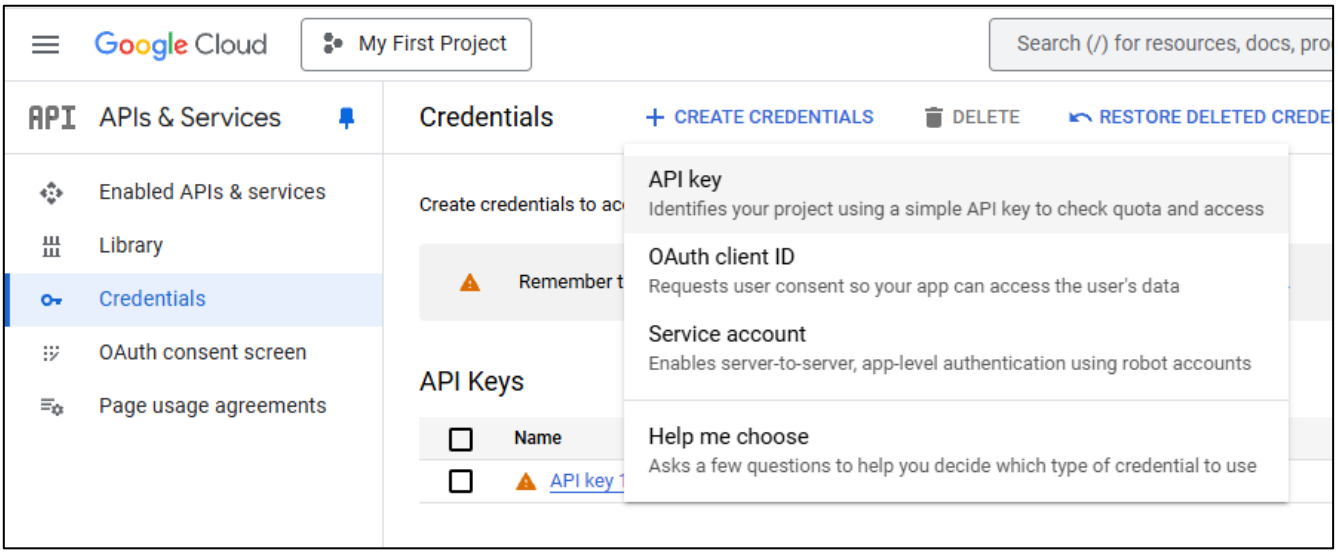


Step 5: After creating the project successfully, it will redirect you to below screen. Now you have to enable Maps Javascript API. Click on Enable APIS & Services → Search for maps javascript api → Enable it.

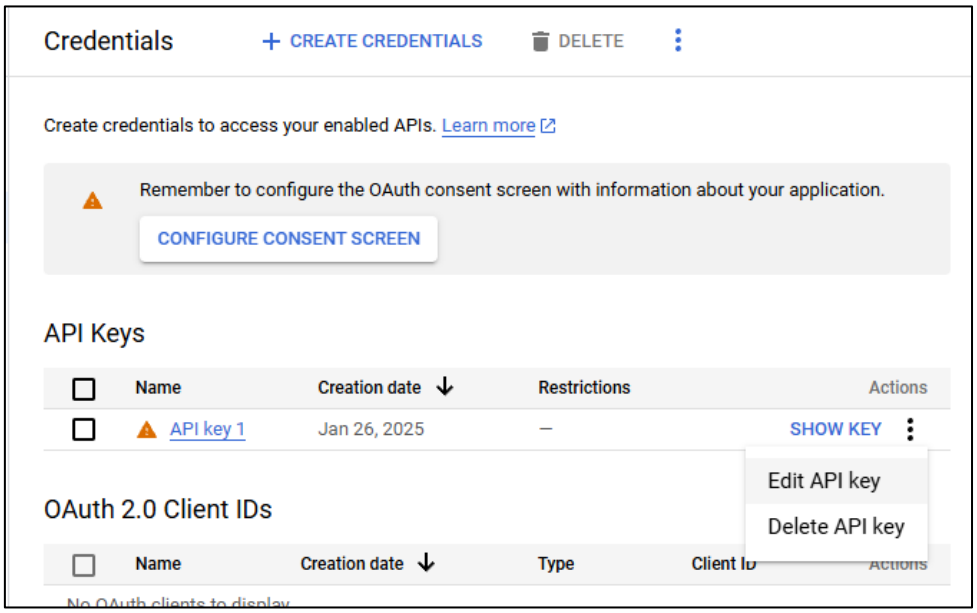




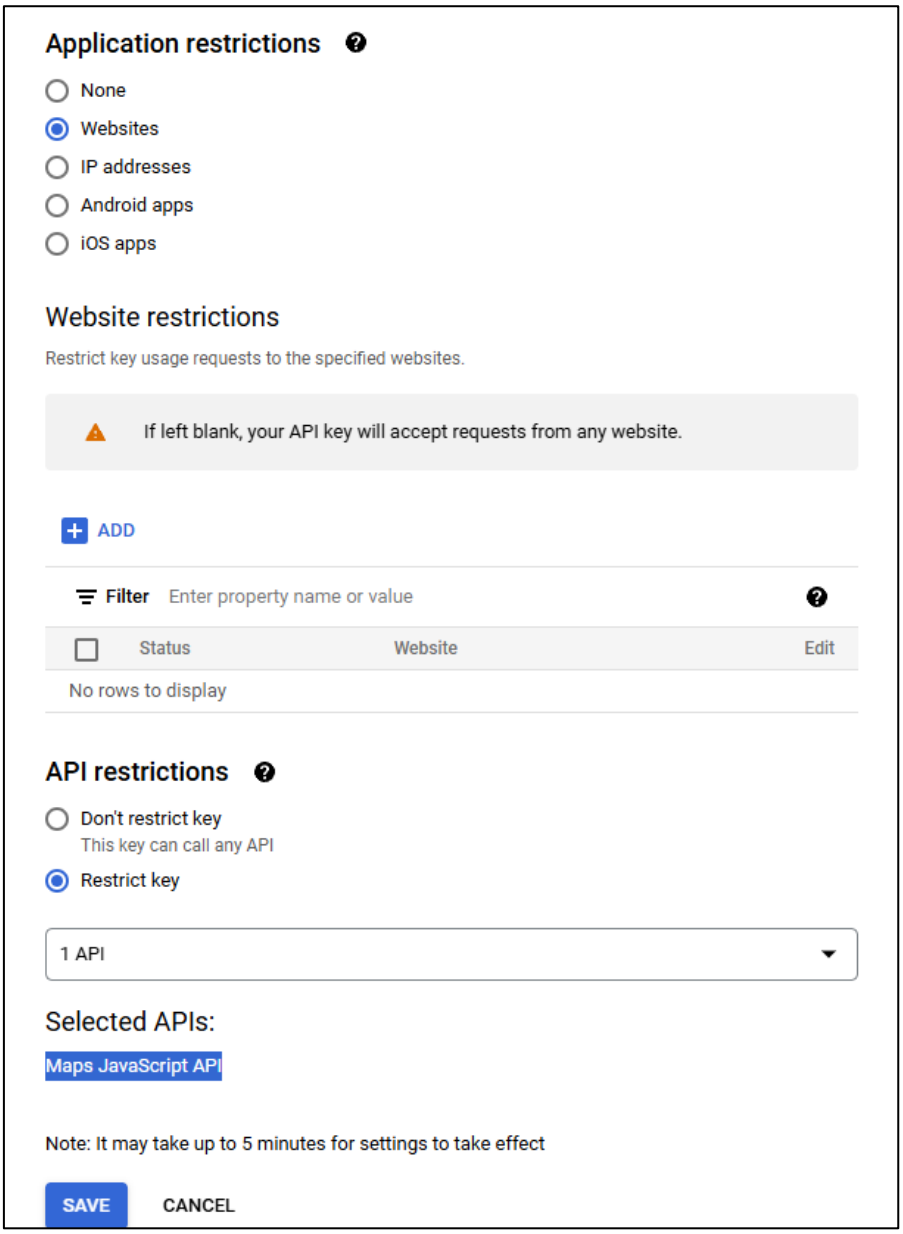
Step 6: Agree & Continue in Account Info Step. Next Leave Verify your Identity Step and navigate to Credentials Tab on Left Side. Click on Create Credentials and then Click on API Key.



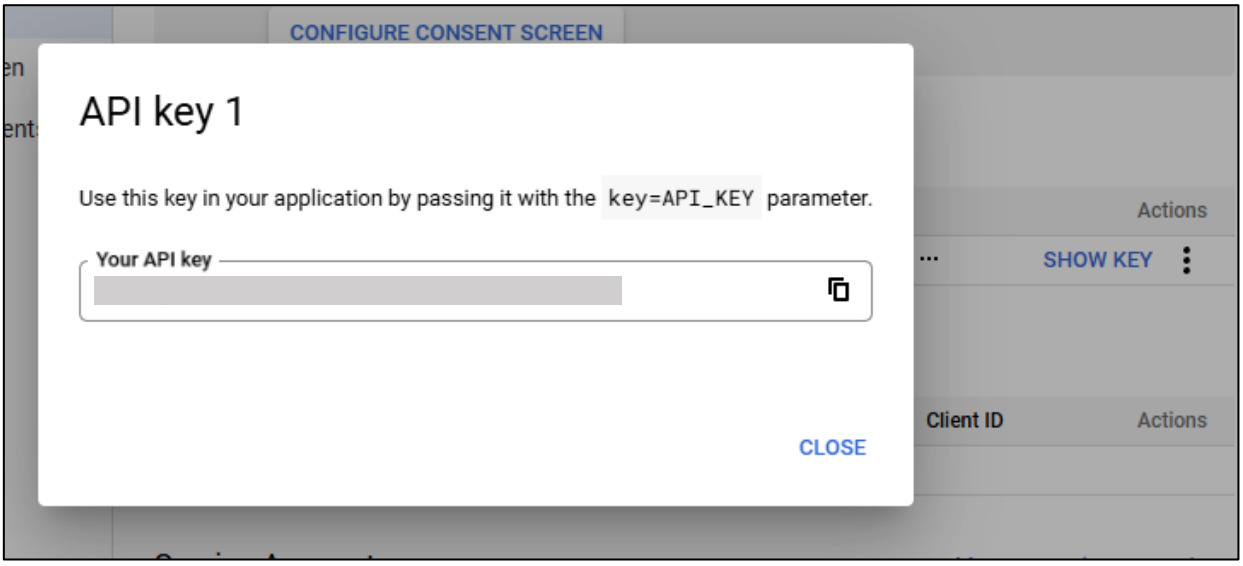
Step 7: Edit your API Key



Step 8: Now, Select **Websites** in App Restrictions Category, Select **Restrict Key** in API Restrictions and Checkmark **Maps Javascript API** in Options. Save it.



Step 9: View your Key.



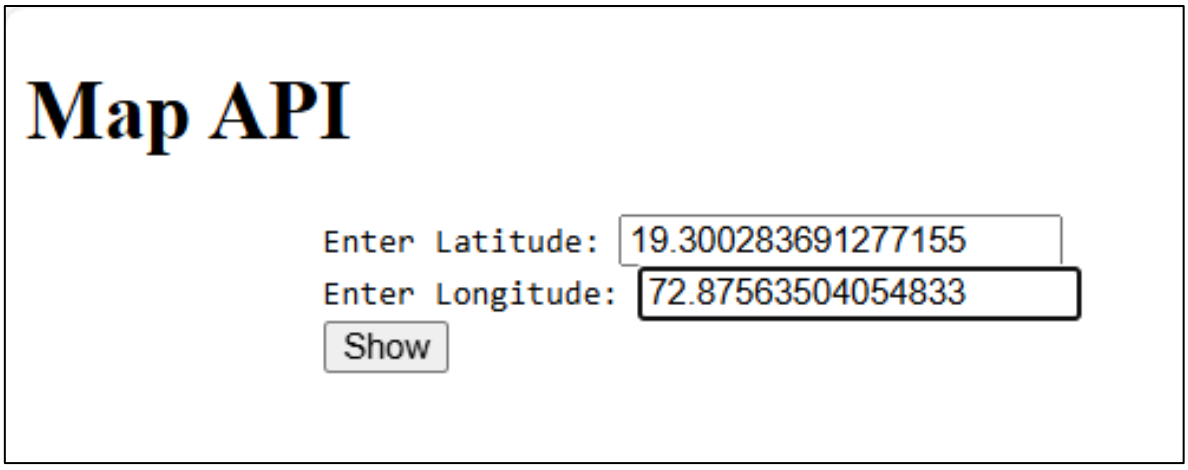
Step 10: Implement foll Code in index.jsp (Paste **your API Key** in Code)

```jsp
<%--
    Document   : index
    Created on : 26 Jan, 2025, 4:07:18 PM
    Author     : Admin
--%>

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>JSP Page</title>
        <style>
            #map{
                height: 400px; /* Increased height for visibility */
                width: 100%;
            }
        </style>
    </head>
    <body>
        <h1>Your Location:</h1>
        <%
            // Parse the latitude and longitude values from the request parameters
            double lati = Double.parseDouble(request.getParameter("t1"));
            double longi = Double.parseDouble(request.getParameter("t2"));
            out.println("Latitude: " + lati + ", Longitude: " + longi); // Debugging output
        %>

        <div id="map"></div>

        <script>
            // This function initializes the Google Map
            function initMap() {
            console.log("initMap function is running!"); // Debugging
            var info = {lat: <%= lati %>, lng: <%= longi %>};
            var map = new google.maps.Map(document.getElementById('map'), {
            zoom: 4,
            center: info
            });
            var marker = new google.maps.Marker({
            position: info,
            map: map
            });
            }
        </script>

        <script async defer src="https://maps.googleapis.com/maps/api/js?key=YOUR_API_KEY&callback=initMap"></script>
    </body>
</html>
```

Step 11: Now Deploy the Project and Run Input.jsp

OUTPUT:

Latitude and Longitude Prompt:



Final Output:

Your Location:

Latitude: 19.300283691277155, Longitude: 72.87563504054833